O'REILLY

Day 2 Overview

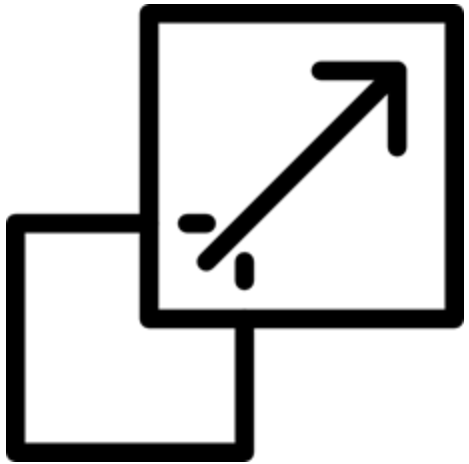
Ammar Mohanna

Day 1 Recap: Foundations of MLOps

- Covered the **MLOps lifecycle**: from data to deployment
- **Containerized** an ML model with **Docker**
- Set up **CI/CD** for automated workflows
- **Deployed** the model via **Flask API**
- **Versioned** models and datasets with **DVC/MLflow**
- Focused on **reproducibility, testing,** and **collaboration**

From Foundations to Real-World Scale

- **Day 1:** Core MLOps (modeling, versioning, deployment)
- **Day 2:** Focus on scaling ML:
  - Kubernetes deployment
  - Handling load and scaling
  - Monitoring and updates
- Introducing new LLMOps concepts



What is Applied MLOps?

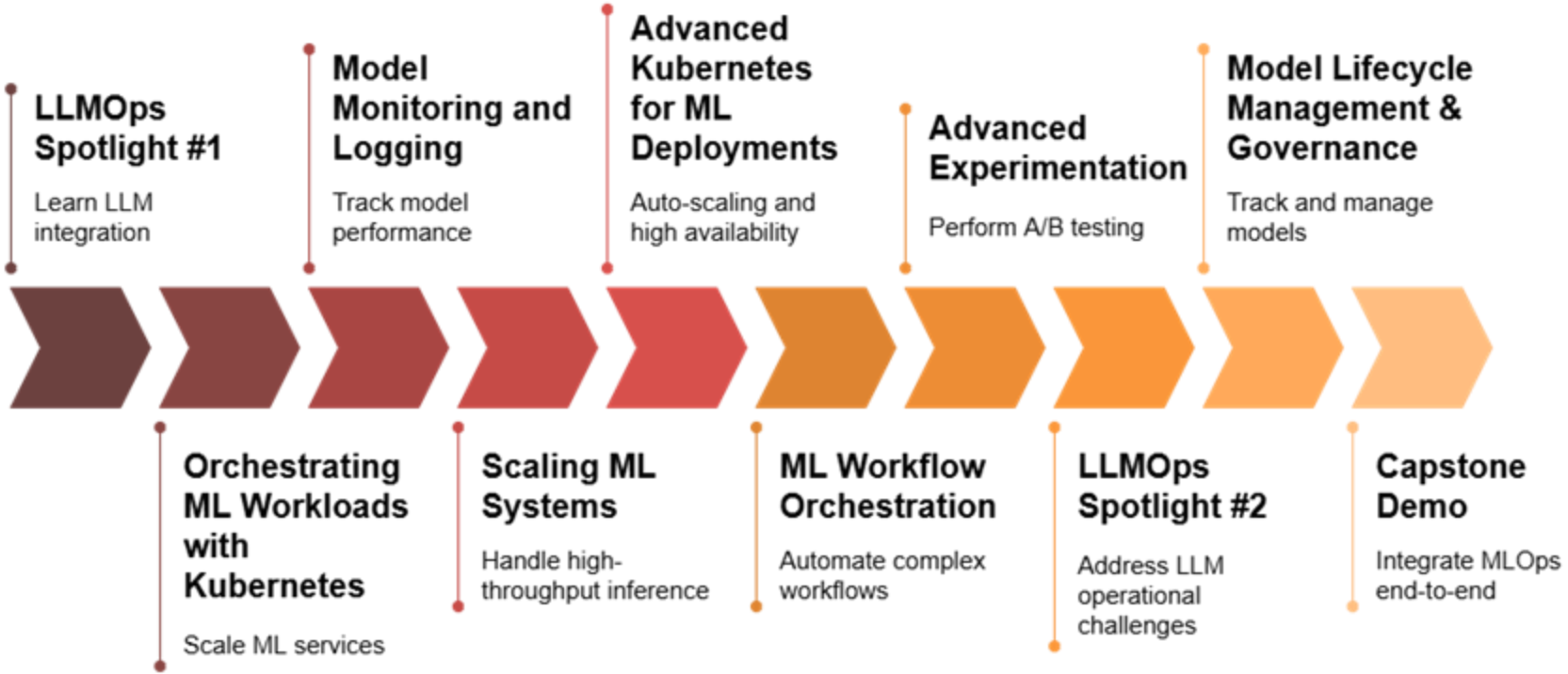
- ML models leave the lab and enter production
- It combines:
  - **Machine Learning**
  - **DevOps**
  - **Software Engineering**
  - **Infrastructure**

Tools We'll Use Today

What You'll Build Today

- Deploy ML model to **Kubernetes**
- Simulate **load** and **auto-scale** with **HPA**
- Monitor **performance** and track key **metrics**
- **A/B test models** with safe rollouts
- Update **ML/LLM pipeline** (retrain → redeploy)

Day 2


LLMOps Spotlight #1
Learn LLM integration
Orchestrating ML Workloads with Kubernetes
Scale ML services
Model Monitoring and Logging
Track model performance
Scaling ML Systems
Handle high-throughput inference
Advanced Kubernetes for ML Deployments
Auto-scaling and high availability
ML Workflow Orchestration
Automate complex workflows
Advanced Experimentation
Perform A/B testing
LLMOps Spotlight #2
Address LLM operational challenges
Model Lifecycle Management & Governance
Track and manage models
Capstone Demo
Integrate MLOps end-to-end

Let’s Get Started


O'REILLY®

O'REILLY

Incorporating LLMs into Your Pipelines

Ammar Mohanna

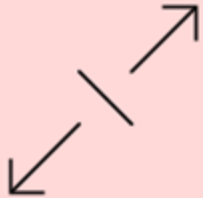
Learning Objectives

By the end of this lesson, you will be able to:

- Explain why teams are **integrating LLMs** into their ML pipelines
- Differentiate **traditional ML** models from **LLM-based workflows**
- Compare **hosted APIs vs. self-hosted LLM** deployment options
- Apply **practical use cases**, like text generation, within a real-world LLM pipeline

LLMs

It All Started with a Transformer



Parallelism

Attention mechanism enables parallel processing of data.

Long-range context

Captures relationships between distant words in sequence.

Massive scalability

Allows training on large datasets and models.

From Research Breakthrough to Product Gold Rush

- ChatGPT marked the tipping point
  - LLMs became **accessible**
  - Users could **talk** to AI
  - Devs wanted to **embed** LLMs in apps
- How do we **integrate** this into our stack? …

LLM Integration — From Idea to Pipeline

Text Generation

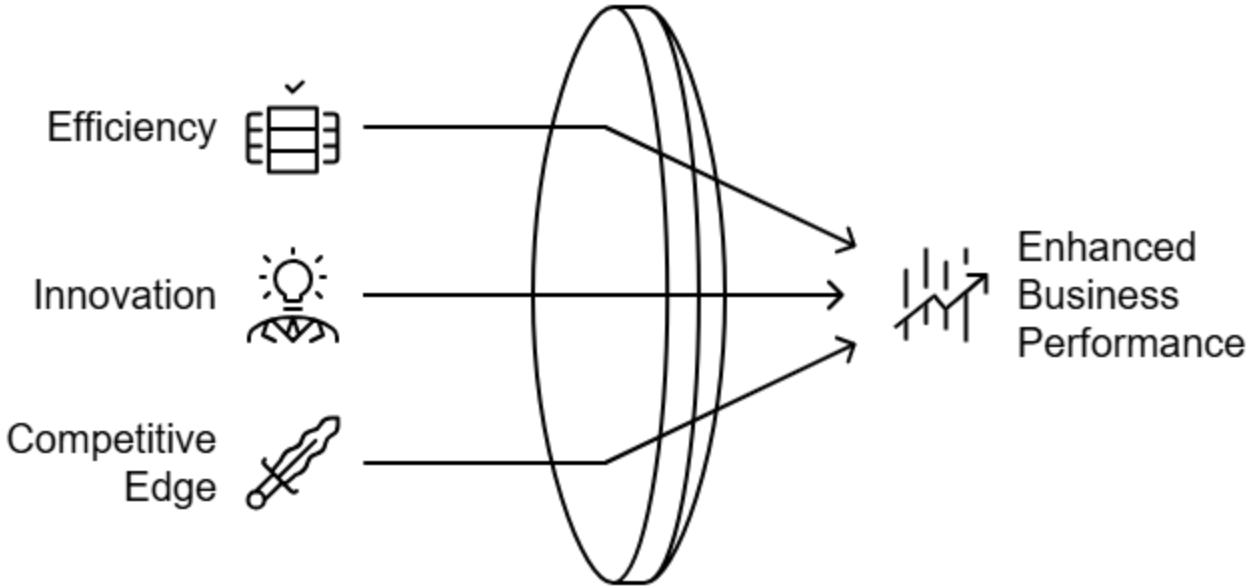
Smart Search

Document Summarization

Content Creation

Code Assistance

Why Teams Are Adding LLMs to Pipelines



Traditional ML vs LLM in Pipelines

Feature	Traditional ML	LLMs
Input	Structured data	Natural language/text
Output	Labels/numbers	Human-like text
Inference	Lightweight (CPU)	Heavy (GPU required)
Training	Model-specific	Few-shot or fine-tune

Key Players in the LLM Space

MISTRAL AI_

Gemini

OpenAI

deepseek

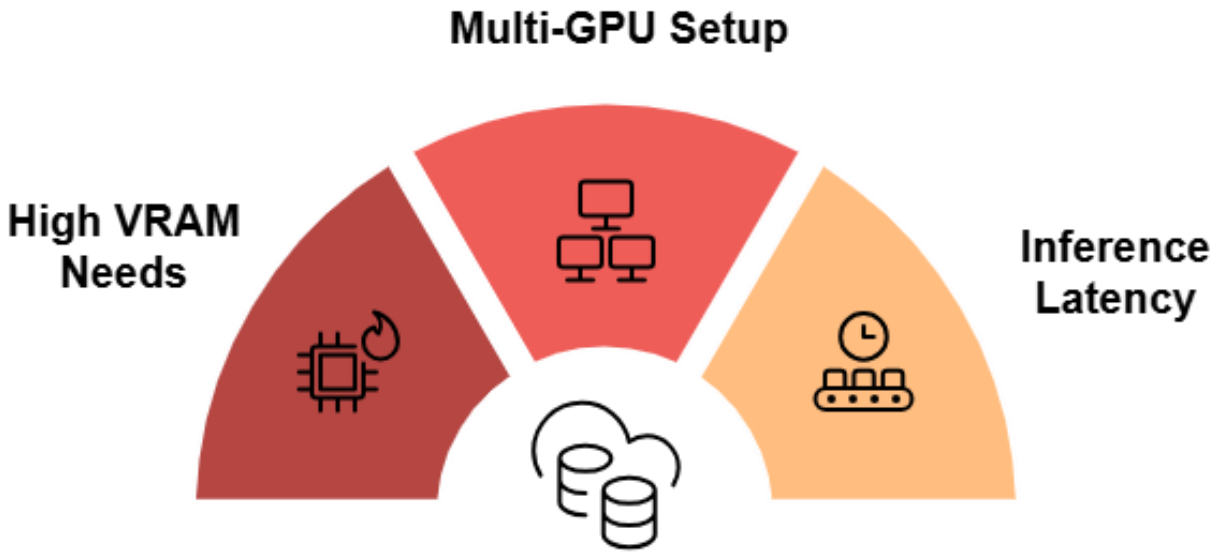
Integrating LLM

Key Considerations

Large Model Checkpoint

Key Considerations

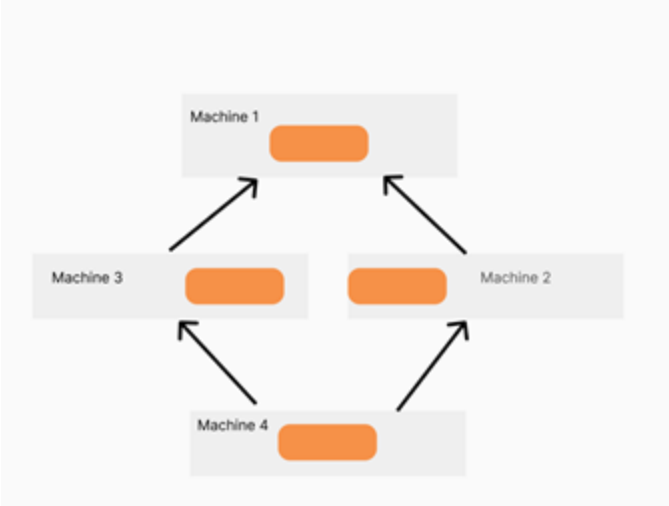
GPU Requirements



Key Considerations

Additional Considerations

- **Model Parallelism:** Necessary for large models across multiple GPUs
- **Energy Consumption:** High operational costs and energy usage for large models



How Teams Start: Hosted APIs

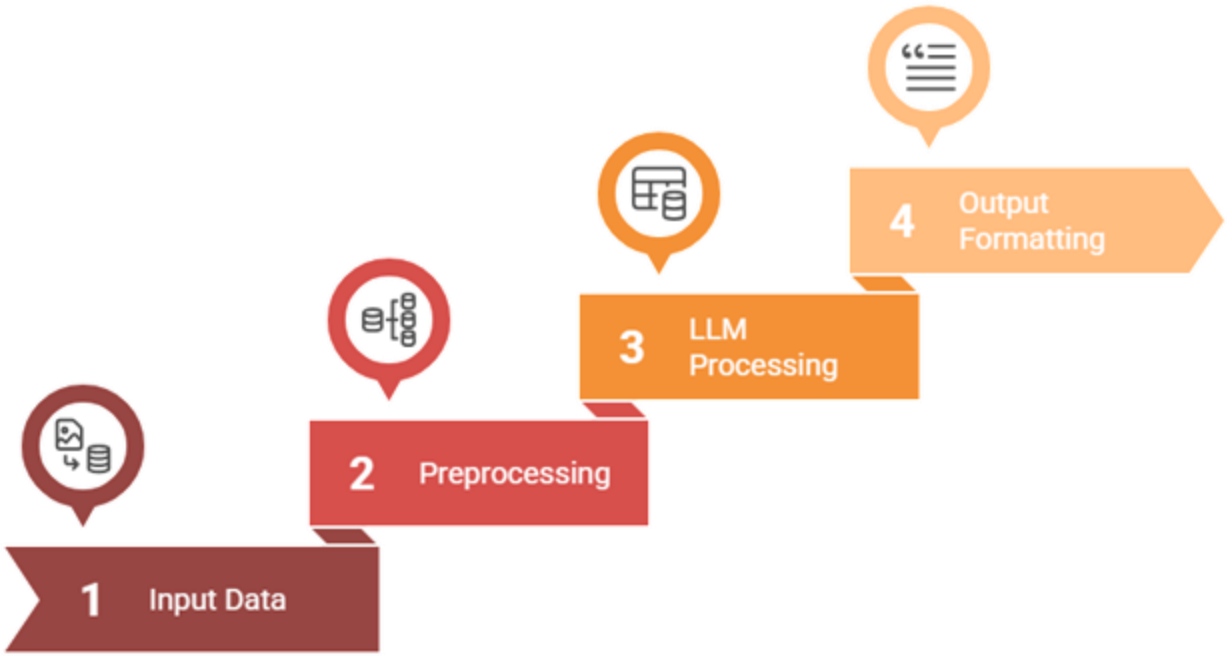
- Fastest way to prototype : OpenAI, Anthropic …
- **Pros:** Easy to use
- **Cons:** Limited control, privacy, cost

Going Deeper — Self-Hosting LLMs

- Hugging Face Transformers,Ollama (local LLMs)
- **Requires:**
  - GPUs
  - Model checkpoints
  - Quantization (for smaller footprints)

HUGGING FACE

Real Pipeline Architecture



Example: Marketing Text Generation for E-Commerce

Pipeline: Turns product data into engaging descriptions

1. **Input**: Gather title, features, audience, tone
2. **Preprocessing**: Build a prompt using a set template

```
Write a fun, modern product description for the following:
Product: Wireless Noise Cancelling Headphones
Features: Bluetooth 5.0, 30-hour battery life, over-ear, lightweight
Audience: Young tech-savvy shoppers
Tone: Playful
```

Example: Marketing Text Generation for E-Commerce

3. LLM(API or Hosted): Generates on-brand, tailored copy

4. Output Formatting: Clean, trim, and prep for output

Summary

Key Takeaways

- **LLMs are now core components**, not just tools—treat them like ML models
- **Hosted APIs are easy to start**, but self-hosting gives you control
- **LLMs need serious infra** such as GPUs, memory, and smart scaling
- **Real value comes from integration**—tie LLMs into real pipelines like marketing, support, or search


O'REILLY®

O'REILLY

Orchestrating ML Workloads with Kubernetes

Ammar Mohanna

Learning Objectives

By the end of this lesson, you will be able to:

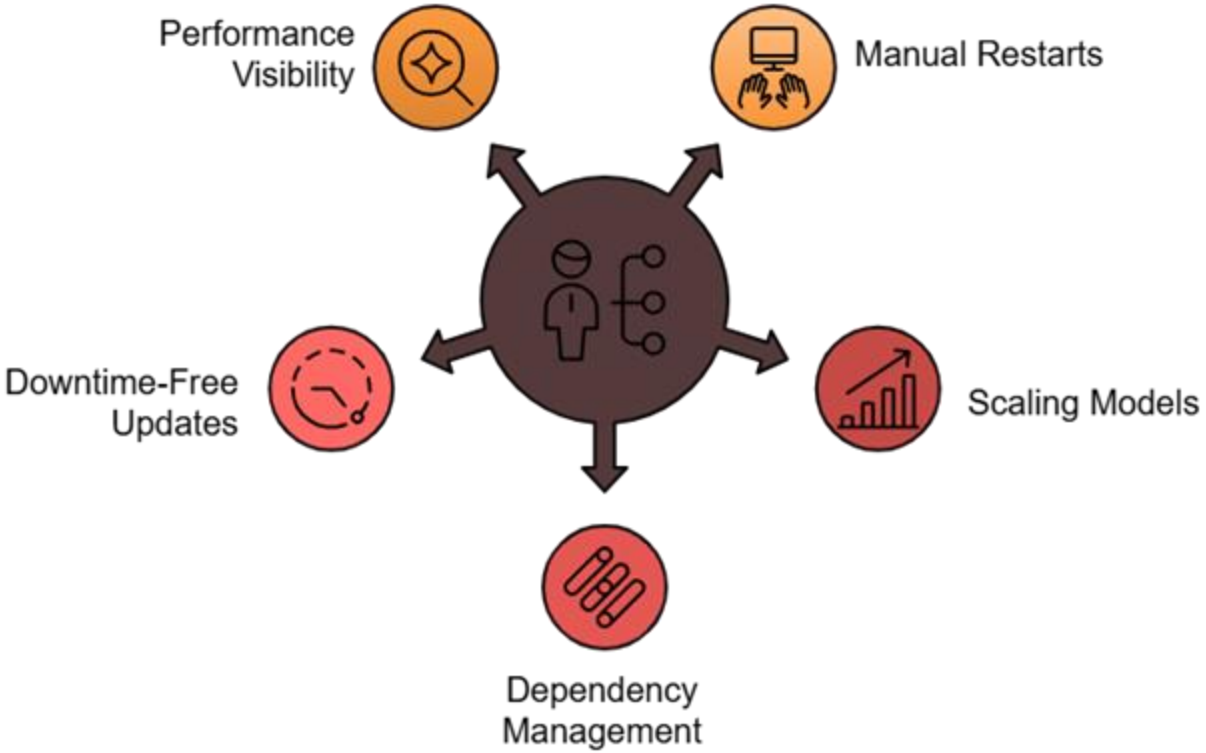
- Understand the **challenges** of deploying ML models in production environments
- Explain how **Kubernetes** helps manage and orchestrate **ML workloads**
- Describe **Kubernetes architecture** and its **key components**
- Recognize the **critical challenges** and considerations when using Kubernetes for ML

Kubernetes

You've trained the model... now what?

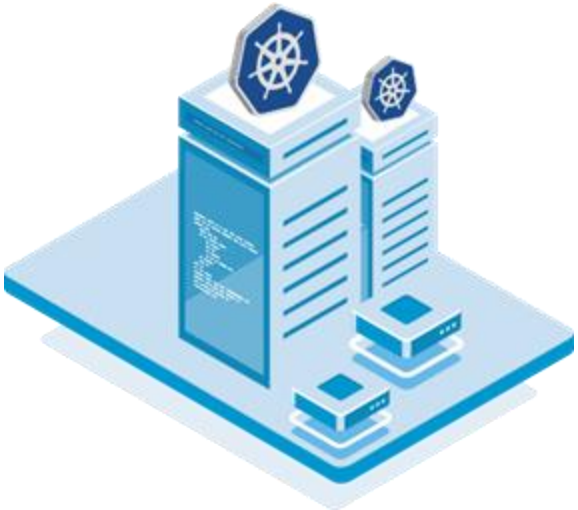
- It works in your notebook …
- But **production**? That's a whole new beast
- **Kubernetes** helps solve this by acting as a powerful **orchestrator**

Common Production Challenges

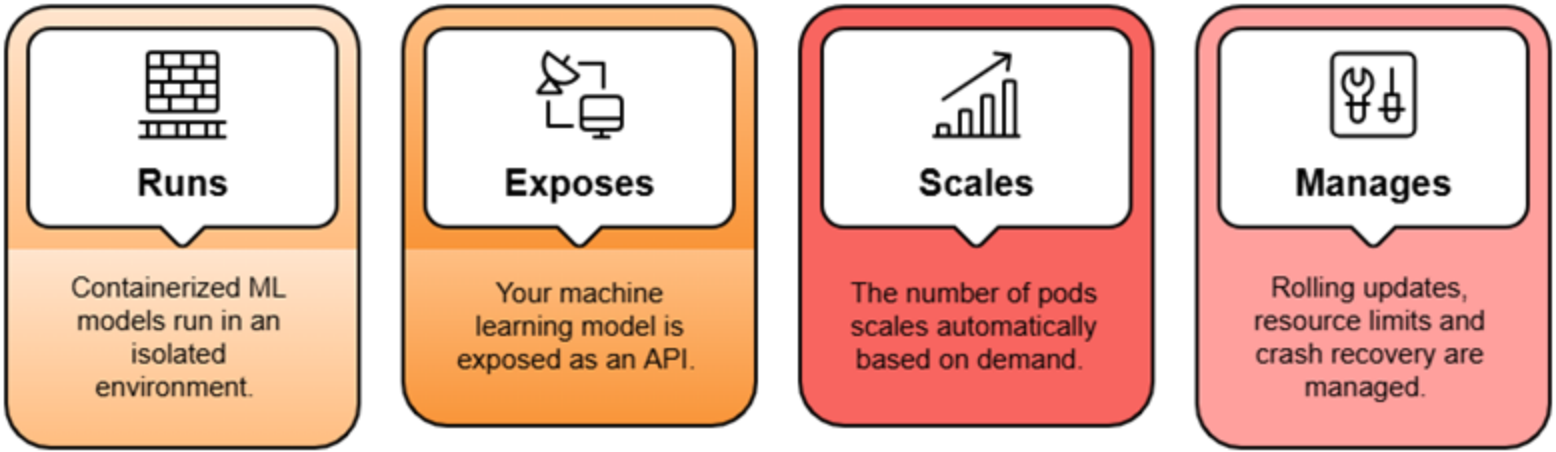


What is Kubernetes (K8s)?

- Simplifies **deployment**, **scaling**, and **management** of containers
- Manages distributed **containerized** systems efficiently



What Kubernetes Does for ML



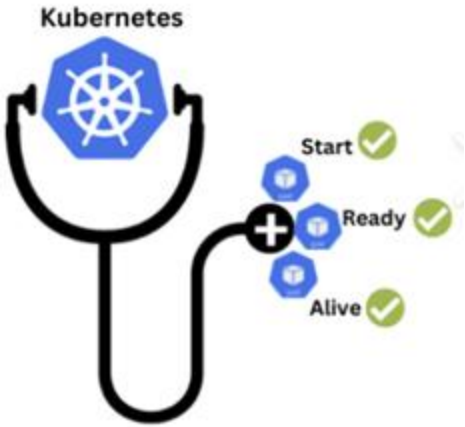
Key Characteristics & Challenges

Portability

- Runs seamlessly **on-premises**, in the **cloud**, or in **hybrid** environments
- Ensures **consistent deployments** regardless of the platform

Self-Healing

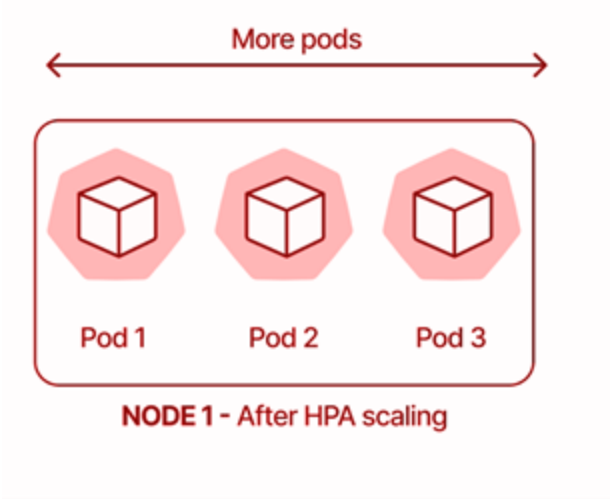
- Continuously **monitors** the health of containers
- Automatically **restarts** failed containers or **reschedules** them on healthy nodes



Scaling: HPA and Resources

- **Horizontal Pod Autoscaler (HPA)**:
  - Automatically scales pods up/down based on metrics
  - For ML: can be extended to custom metrics

- **Resource Requests & Limits:**
  - Ensure pods get appropriate CPU/GPU/memory
  - Prevent noisy neighbors or overloading nodes



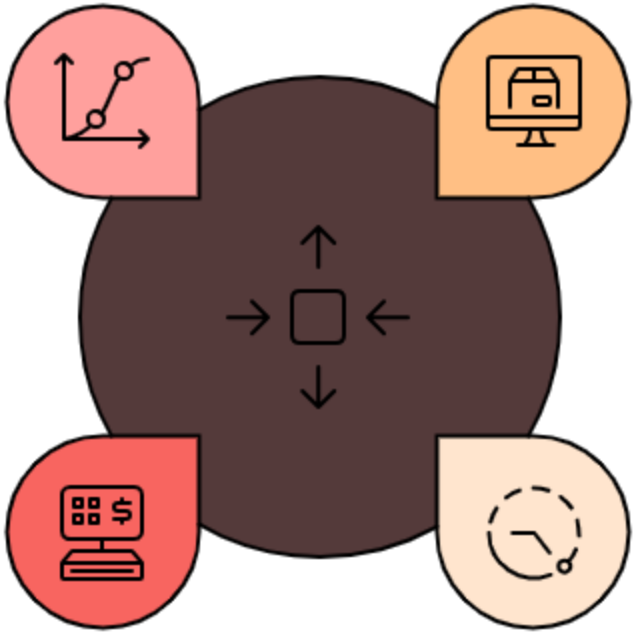
Key Challenges

Complex Metrics

Difficulty tracking latency, queue length, and throughput

Large Model Sizes

LLMs exceeding memory limits on typical nodes



GPU Scheduling and Cost

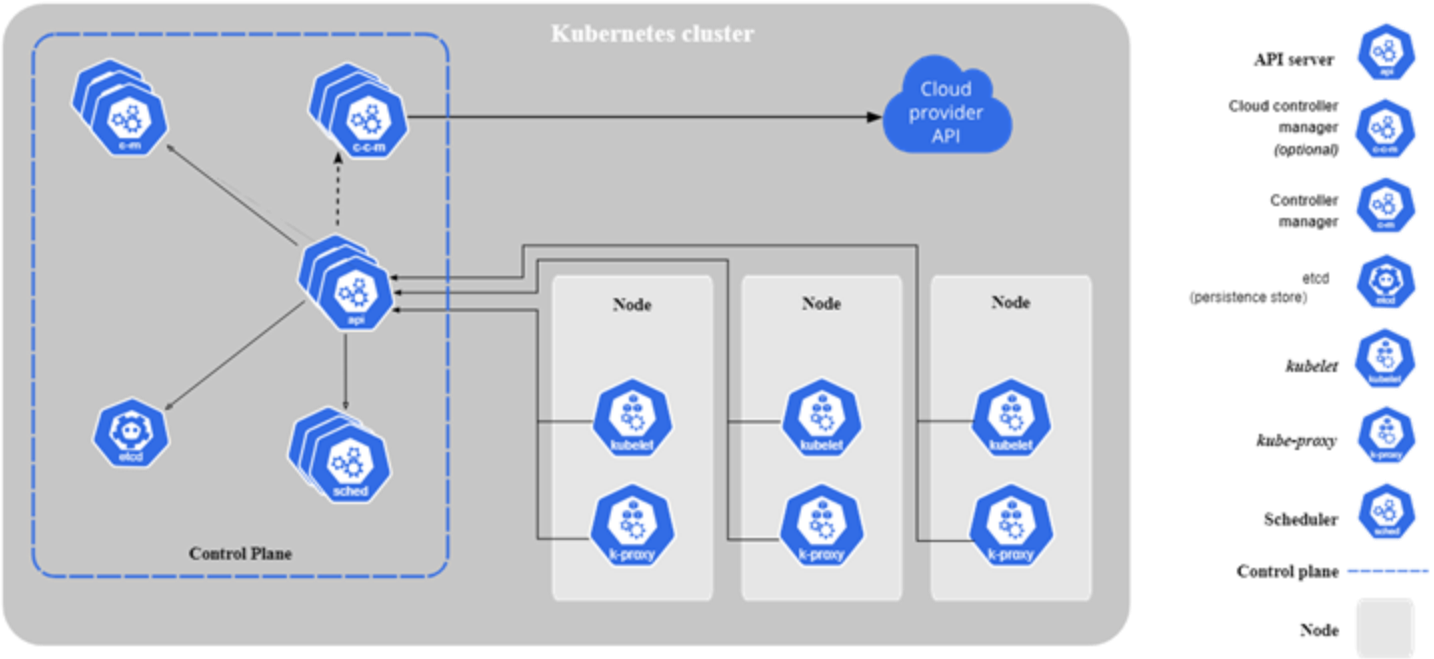
Limited GPU nodes and expensive scaling

Cold Start Latency

Time taken to load models, slowing autoscaling

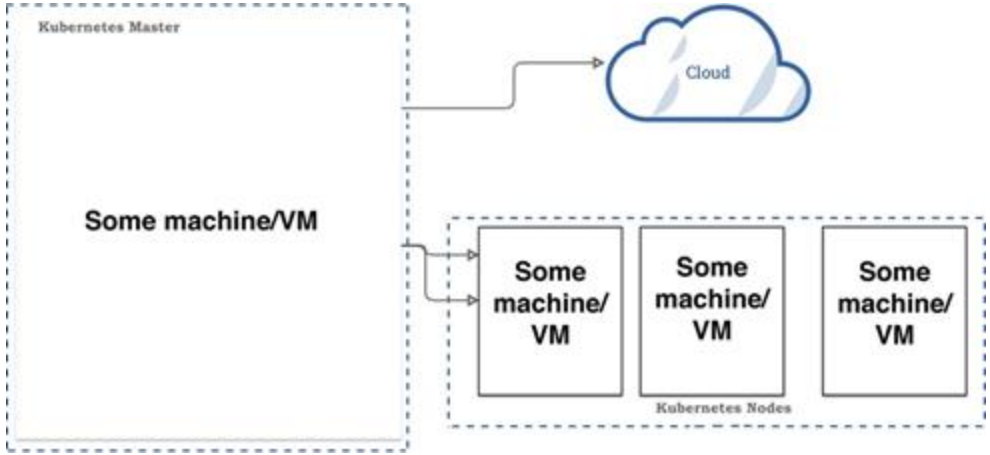
Kubernetes Architecture

Kubernetes Architecture



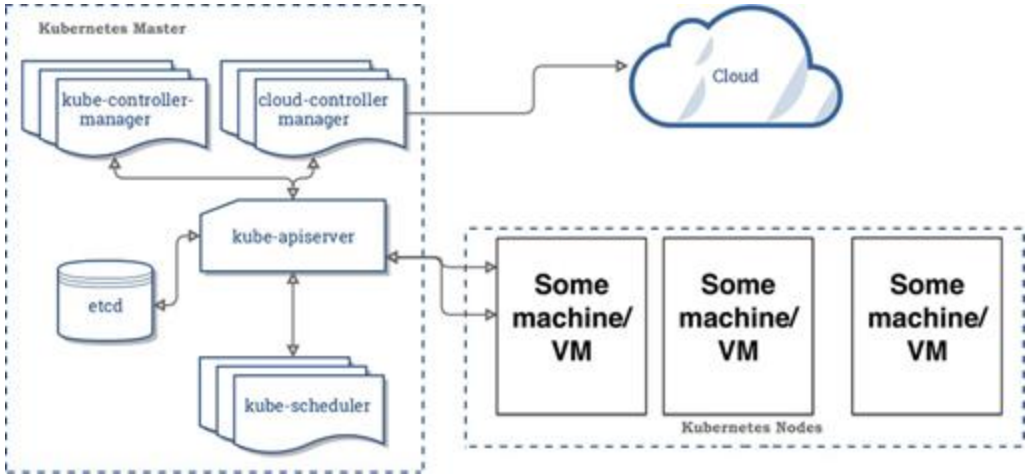
Cluster

- **Group** of **machines** working together
- Forms the **foundation** of a Kubernetes environment
- The cluster includes one **master node** and at least **one worker node**



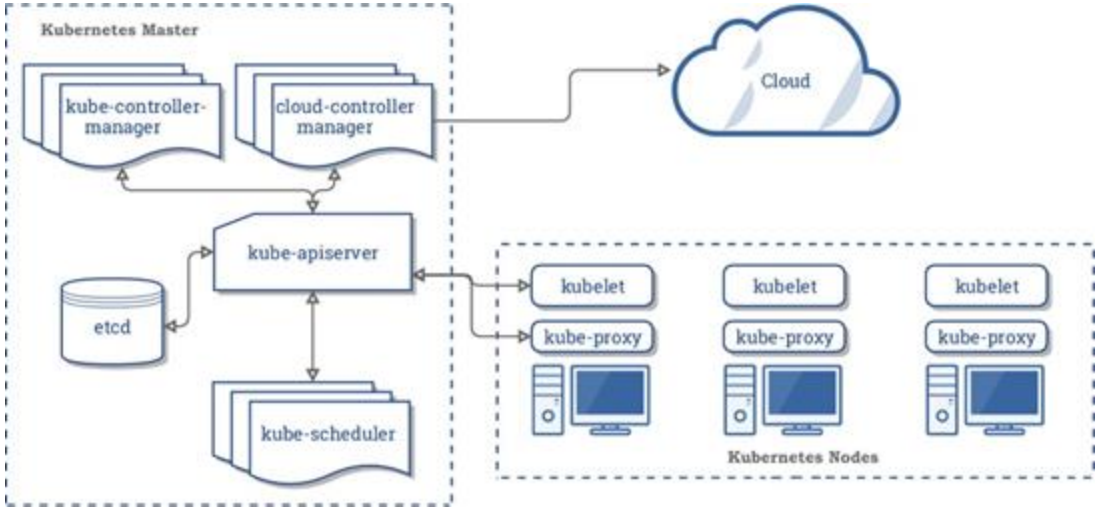
Kubernetes Master Node

- **Controls the Cluster:** Manages and coordinates the cluster's activities
- **API Server:** Handles requests and communication within the cluster
- **Scheduler:** Assigns workloads to worker nodes



Kubernetes Worker Node

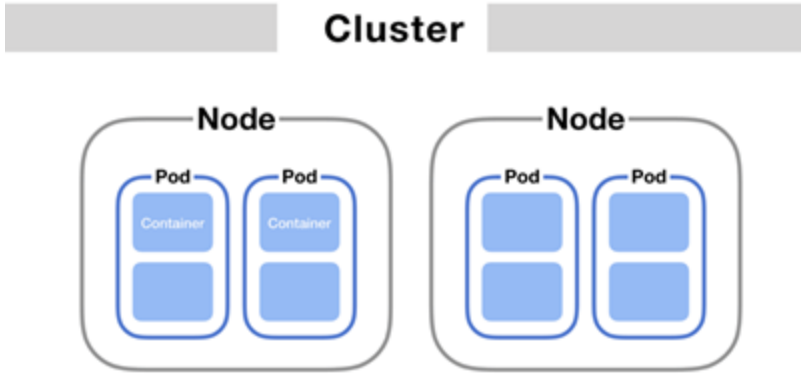
- **Runs Applications:** Hosts and executes containers (pods)
- **Kubelet:** Ensures containers are running as expected



Kubernetes Core Components

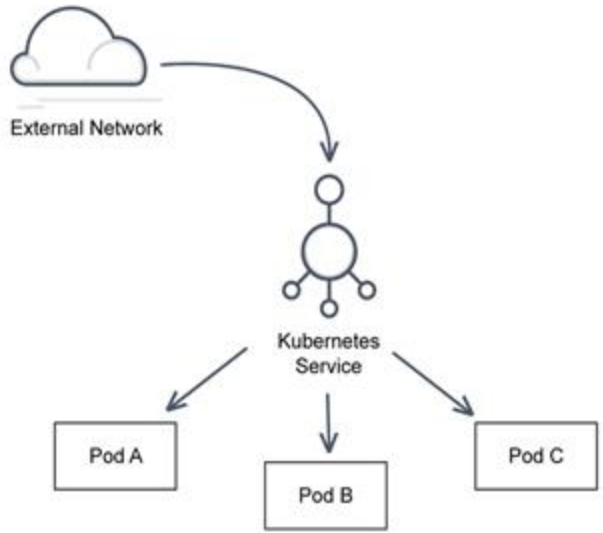
Pods

- A **pod** is the smallest deployable unit in K8s
- Usually running one container (your ML model)
- **Example:**
  - Flask/FastAPI serving a scikit-learn model



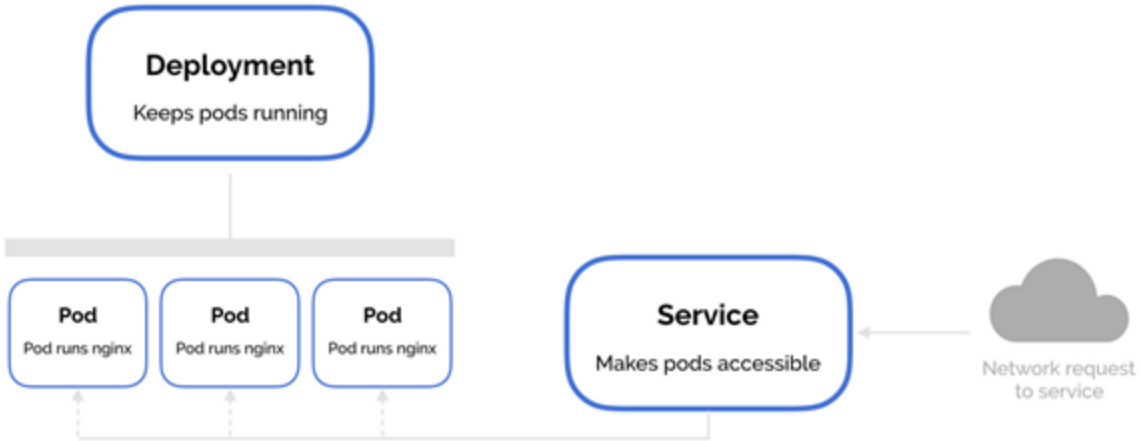
Services

- Services let you **expose** your pod to other services or external clients
- **Pod Communication:** Enable communication between Pods



Deployment

- Manage application changes through **YAML** or **JSON**
- Ensure **consistency** during:
  - scaling
  - rolling updates
  - rollbacks



Summary

Key Takeaways

- **Kubernetes** simplifies container deployment, scaling, and management
- The **Kubernetes architecture** includes a master node (API server, scheduler)
- **Core Components**:
  - **Pods**: Smallest unit, runs the ML model
  - **Services**: Handle communication between pods
  - **Deployments**: Manage scaling, updates, and rollbacks

kubernetes

Pulse Check

What's one challenge when scaling an LLM-based service in K8s?

Hands-on: Deploying Age Detection Model using K8s

Break


O'REILLY®

O'REILLY

Model Monitoring and Logging

Ammar Mohanna

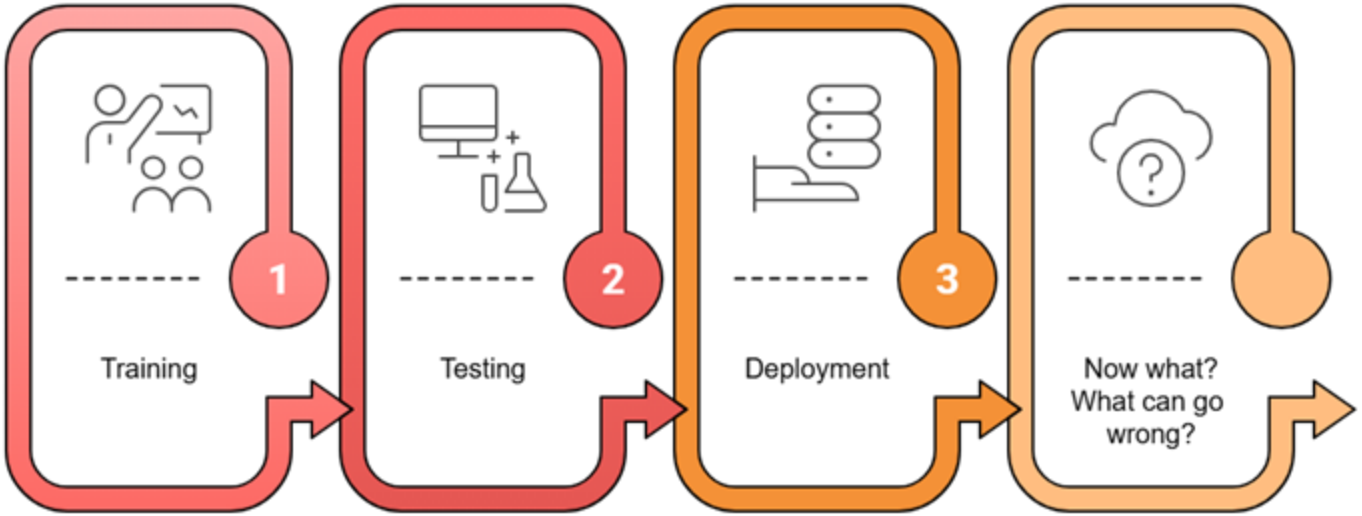
Learning Objectives

By the end of this lesson, you will be able to:

- Understand the unique **monitoring** needs of **ML** and **LLM** systems
- Identify what **components** to monitor at the system, data, model, and business level
- **Detect** and **diagnose** model performance issues over time
- Use **logging, visualization,** and **alerting** effectively for ML systems
- Apply **best practices** to build a resilient monitoring and logging setup

Challenges in ML Systems

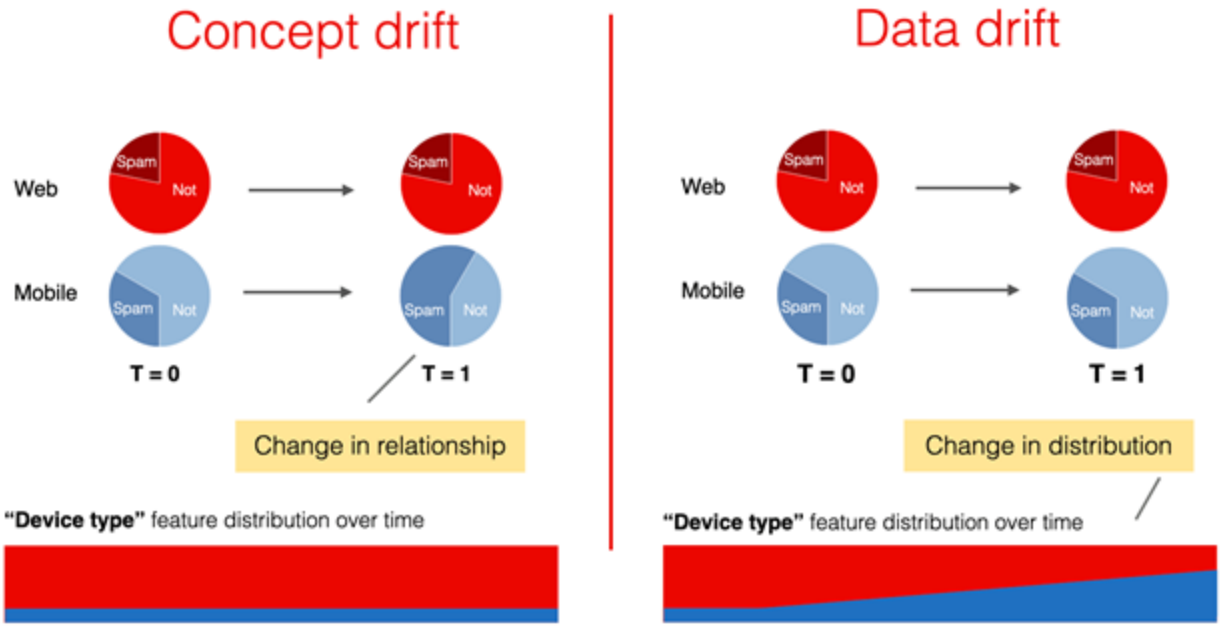
Challenges in ML Systems



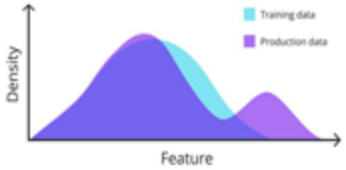
Model Performance Degradation

- Models don’t fail overnight — they **drift** quietly
- Continual **monitoring** is the only way to stay ahead

Comparison between Concept & Data Drift



ML Specific Failures



Distribution Differences

Production and training data have different distributions.

1

Rare Inputs

The model may encounter rare, unforeseen inputs.

2

Software System Failures

Dependency Failures

Deployment Failures

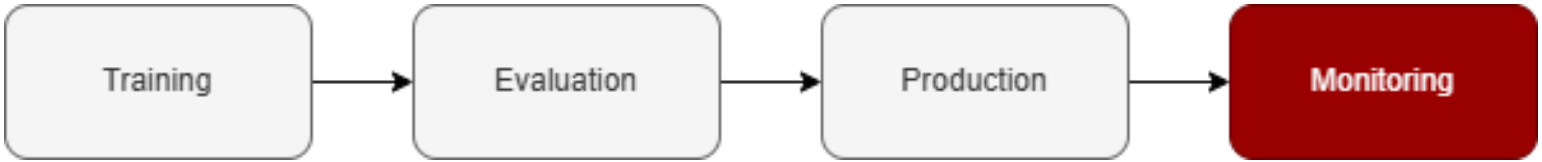
Hardware Failures

System Downtime

Monitoring

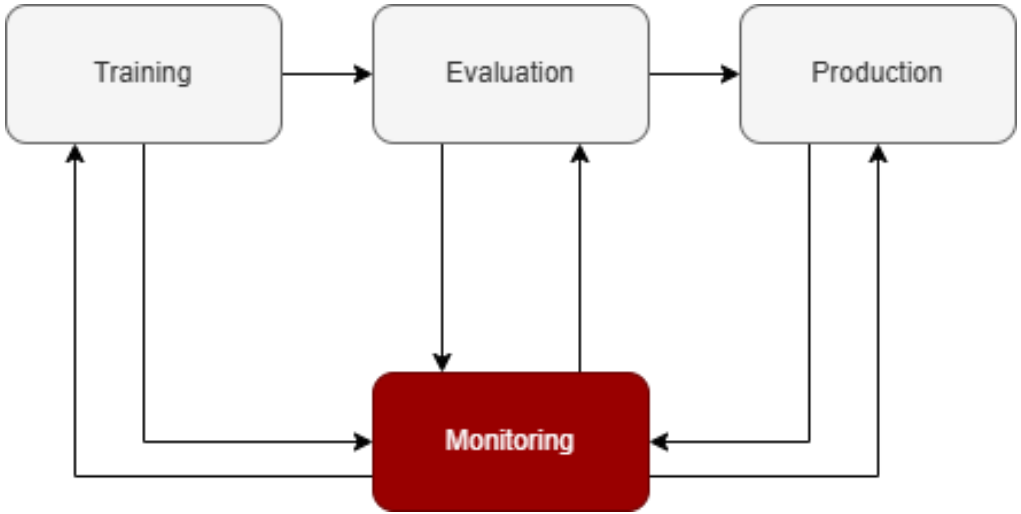
A Traditional View of Monitoring

- Bugs are usually **obvious** and/or cause loud **failures**
- Treat Model as Static Artifact



The Reality of Monitoring in ML

- Bugs usually cause **silent** depredations in performance
- The data you monitor (system metrics, etc) is literally the **code** used to produce the **next model**

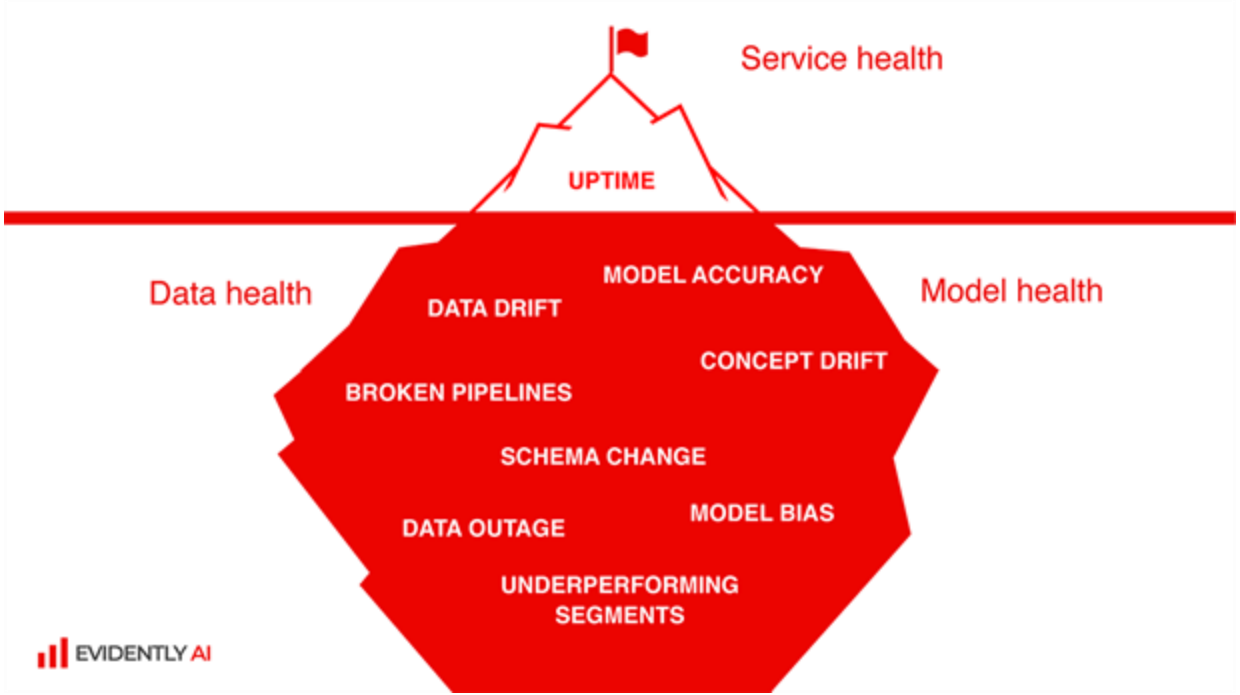


Monitoring

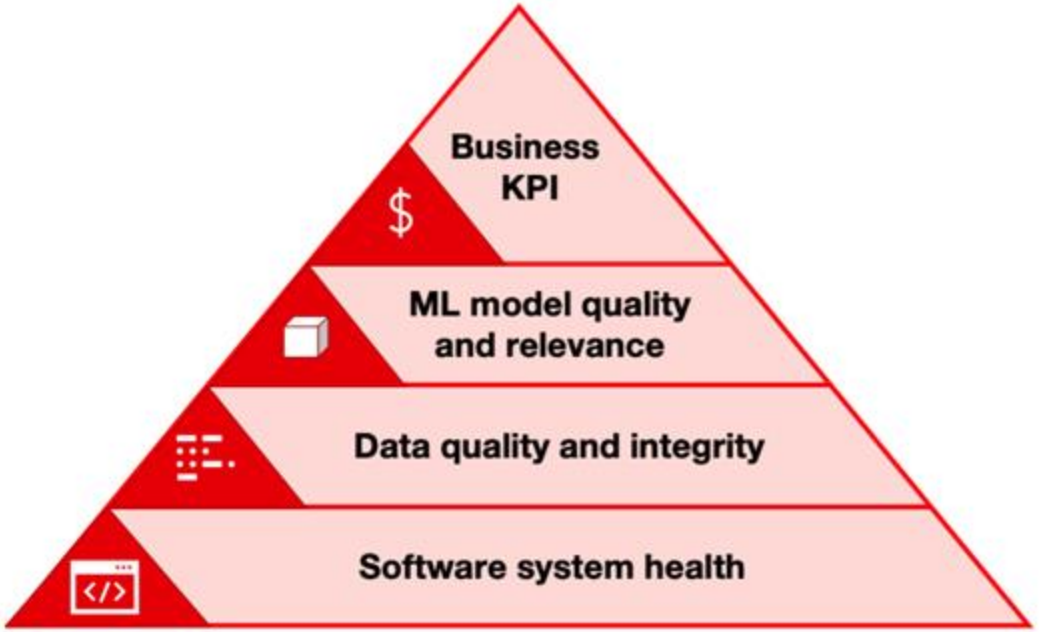
- Continuous **collection** of model-specific metrics
- Real-time **analysis**
- **Logging** of inputs, outputs, and intermediate features for post-hoc diagnostics

What should I Monitor

What can go wrong?



Monitoring Pyramid



Software System Health

- **Monitor:**
  - Availability (Uptime)
  - Performance (Latency, Throughput
  - Reliability (Error Rates, Fault Tolerance)

Got an awesome model

.. but the system is down

Data Quality and Integrity

- **Monitor:**
  - Missing values
  - Schema deviations
  - Unexpected distributions.

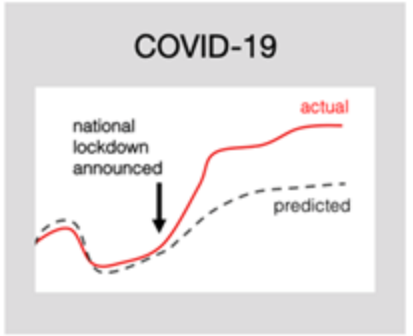
Great service performance

.. but the input data is broken

ML Model Quality and Relevance

- **Monitor:**
  - Model Specific Metrics
  - Concept Drift

Stable pipelines and data



.. but things changed (a lot!)

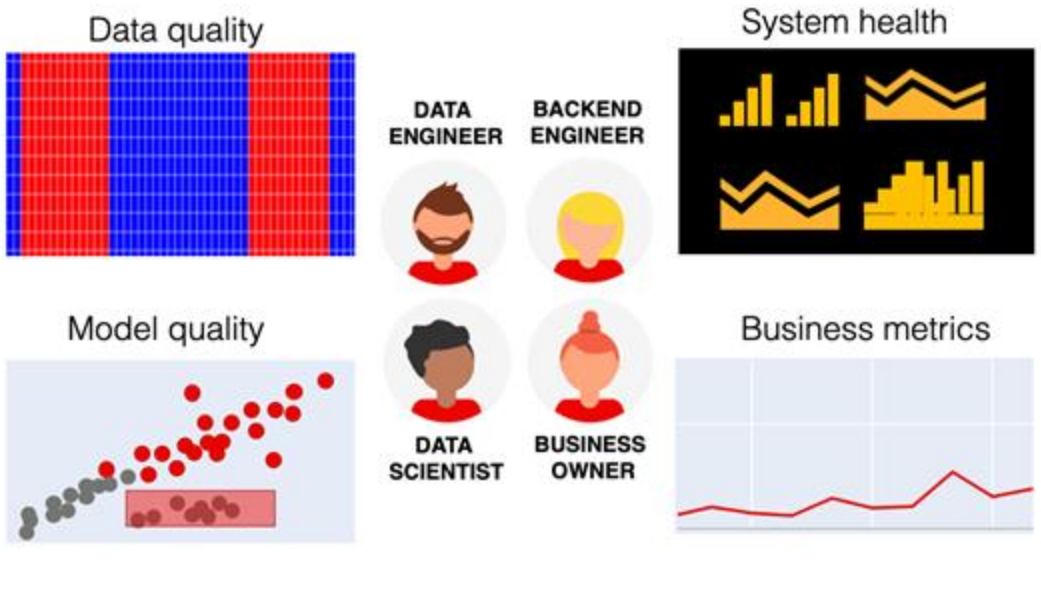
Business KPIs

- Measure **KPIs** based on your **goals**
- Monitor KPI **trends** and set **alerts** for significant deviations

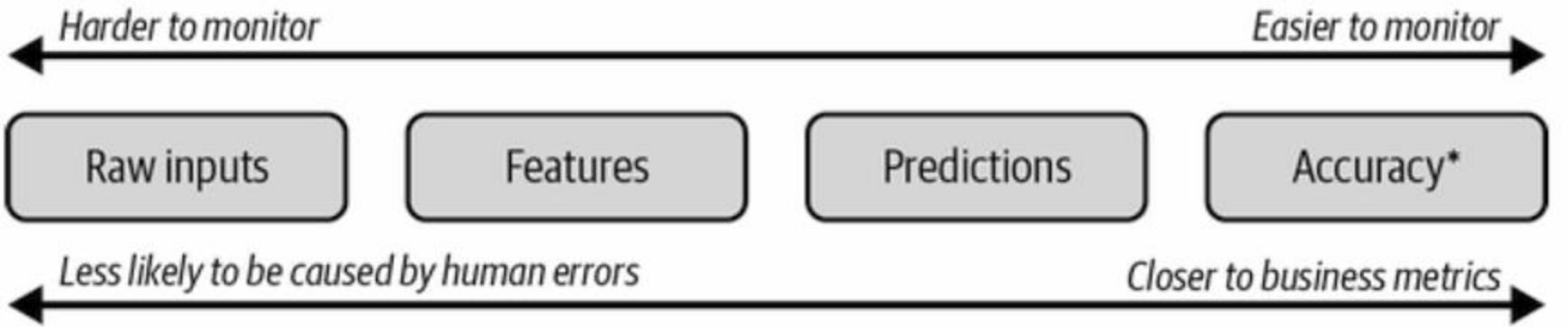
Great production quality

… but the product metrics are down

What Should I Monitor



ML-Specific Metrics



What to Monitor in LLMs

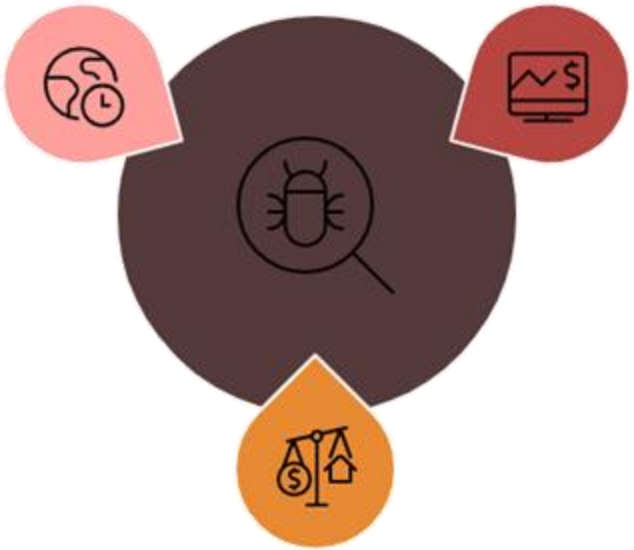
Characteristic	Description
 **Perplexity**	Lower is better
 **Toxicity Scores**	Detects unsafe outputs
 **Hallucination Detection**	Fact-checking or consistency scoring
 **Retrieval Augmentation Metrics**	Context relevance, answer groundedness

How do I measure if it’s changed

Detecting Issues

Time and Spatial Monitoring

Observing shifts over time and across dimensions



Monitoring Performance

Tracking metrics to identify performance changes

Statistical Techniques

Using hypothesis tests to find distribution differences

Reference Window Selection

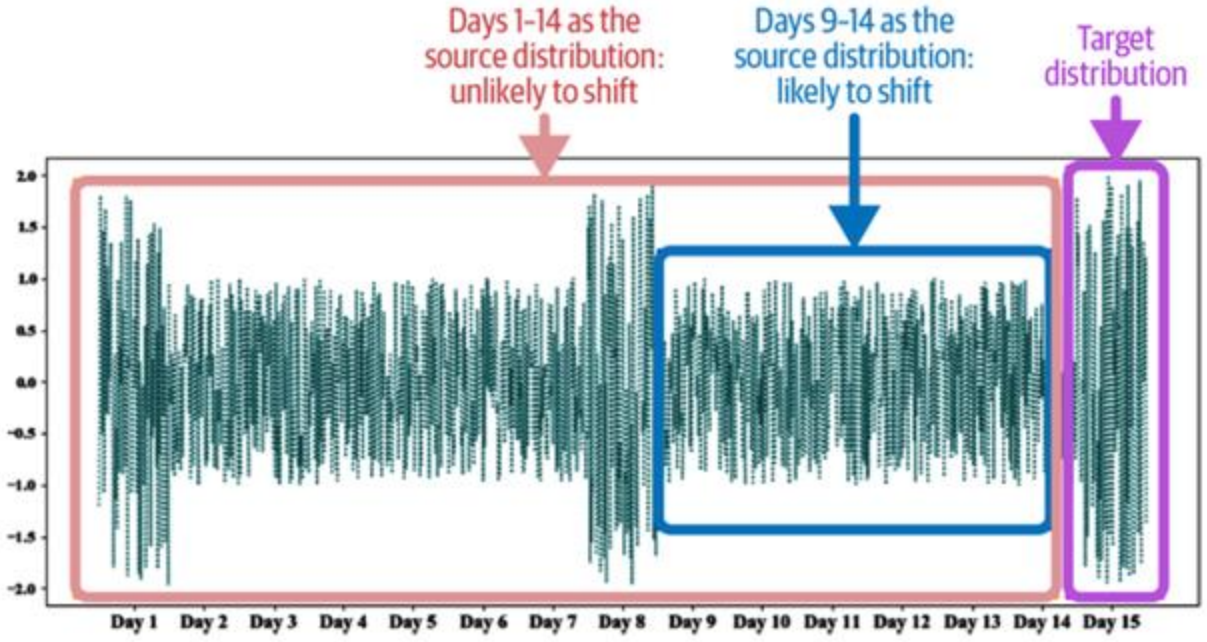
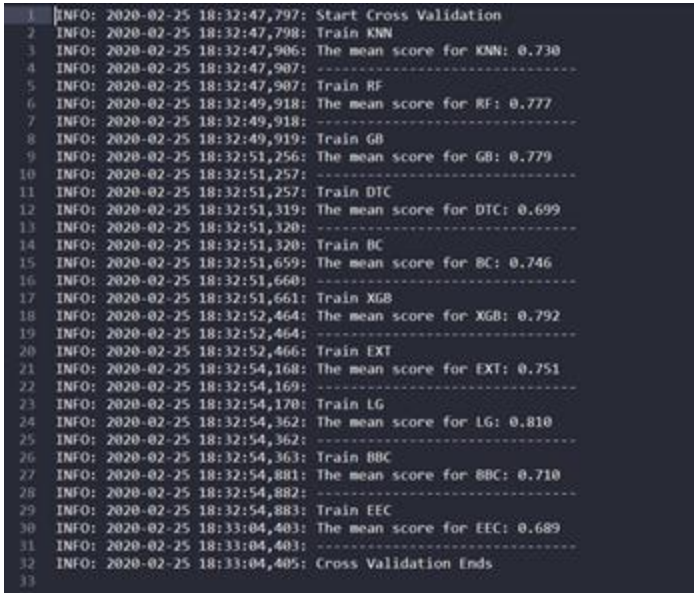


Figure 8-3. Whether a distribution has drifted over time depends on the time scale window specified

Tools for Monitoring

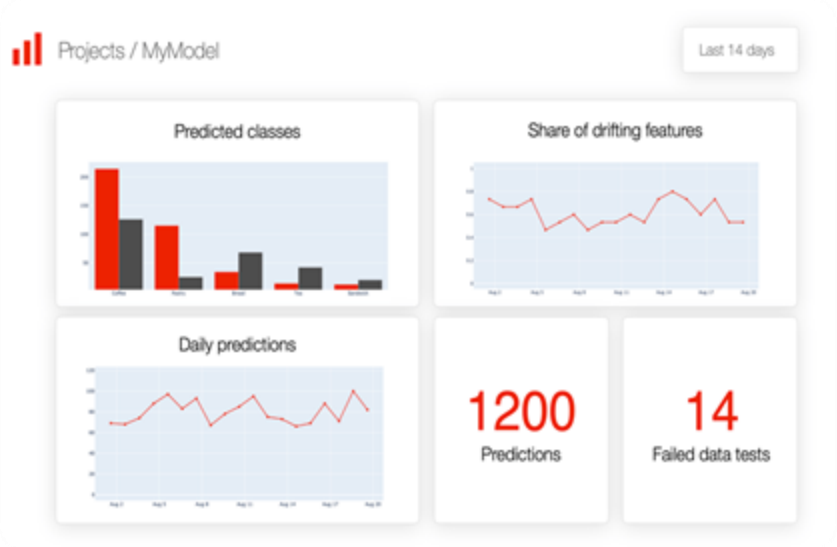
Logging

- Captures model **inputs**, **outputs**, and **internal decisions**
- Enables **traceability** for debugging and audits
- Helps **detect anomalies** and **drift** over time



Visualization

- Detect **performance degradation** early
- Spot **patterns, seasonality,** or **anomalies**
- Understand **relationships** between features and outcomes
- Make monitoring **accessible** to non-technical stakeholders

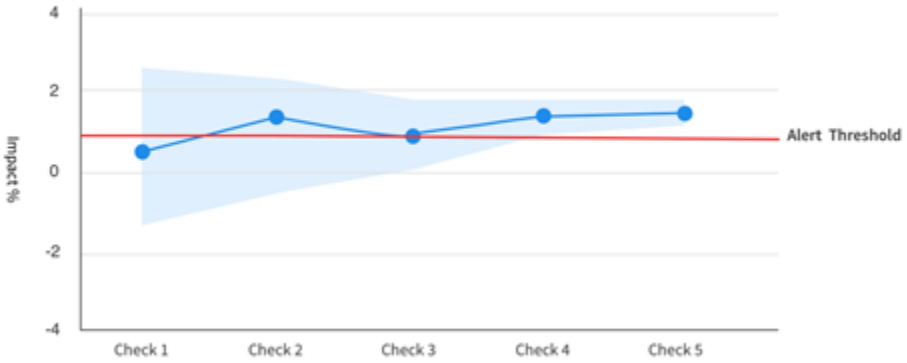


Alerts and Thresholding

- Detect **anomalies** early
- Trigger **investigations** and rapid **responses**
- Aid in **model tuning** and **retraining** decisions

How to set Alerts

- Define clear **thresholds** for when performance is "**bad enough**" to act
  - **Example:** If perplexity increases by 10%, trigger an alert
- Thresholds must **balance** sensitivity and noise



LLM Monitoring Stack

LLMOps Tools

Weights & Biases, Arize AI, Helicone.

Logging Tools

Prometheus, OpenTelemetry, custom JSON logs

Alerting Tools

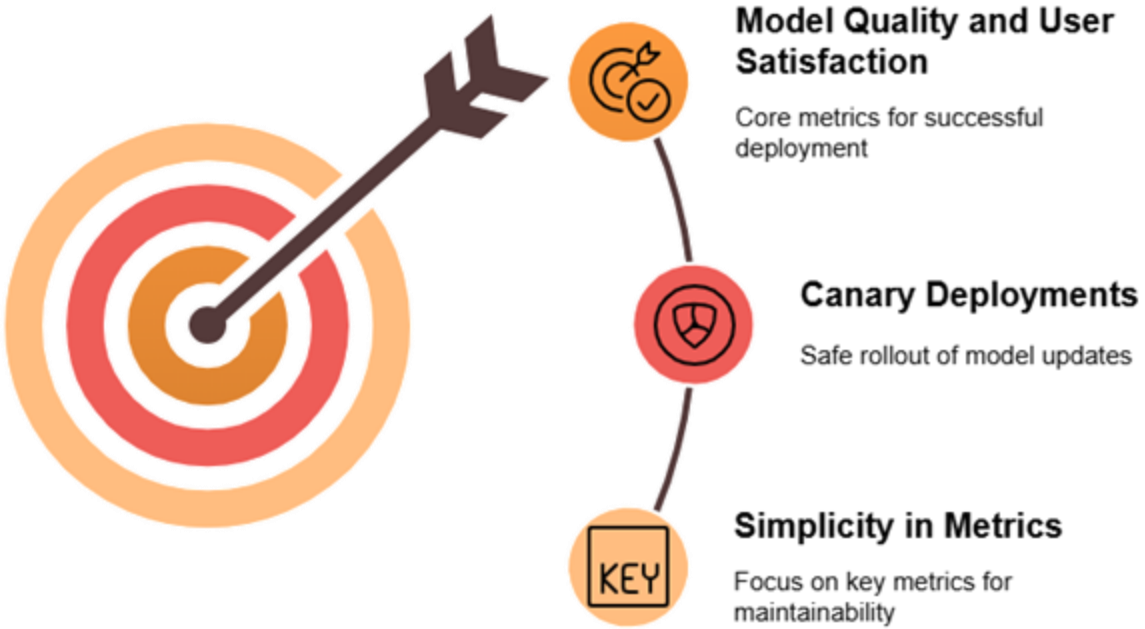
PagerDuty, Opsgenie,Prometheus.

Dashboard Tools

Grafana, Kibana

Best Practices

Best Practices



Summary

Key Takeaways

- Models quietly degrade over time through **concept** and **data drift** — requiring continual monitoring
- Monitor **system health,data quality, model quality,** and **business KPIs**
- **Compare** training vs. inference statistics**,** use **hypothesis testing,** and **monitor** time-based and user-based shifts to catch problems early**.**
- Use **logging, visualization,** and **alerts** to detect and respond quickly.


O'REILLY®

O'REILLY

Scaling ML Systems

Ammar Mohanna

Learning Objectives

By the end of this lesson, you will be able to:

- Grasp the **fundamental principles of ML scalability** and why it matters in real-world systems
- Identify key **scaling challenges** such as large data volumes, complex models, and high inference demand
- Recognize **solutions** and **strategies** for scaling
- Learn about **LLM-specific scaling** issues like distillation, quantization, and MoE

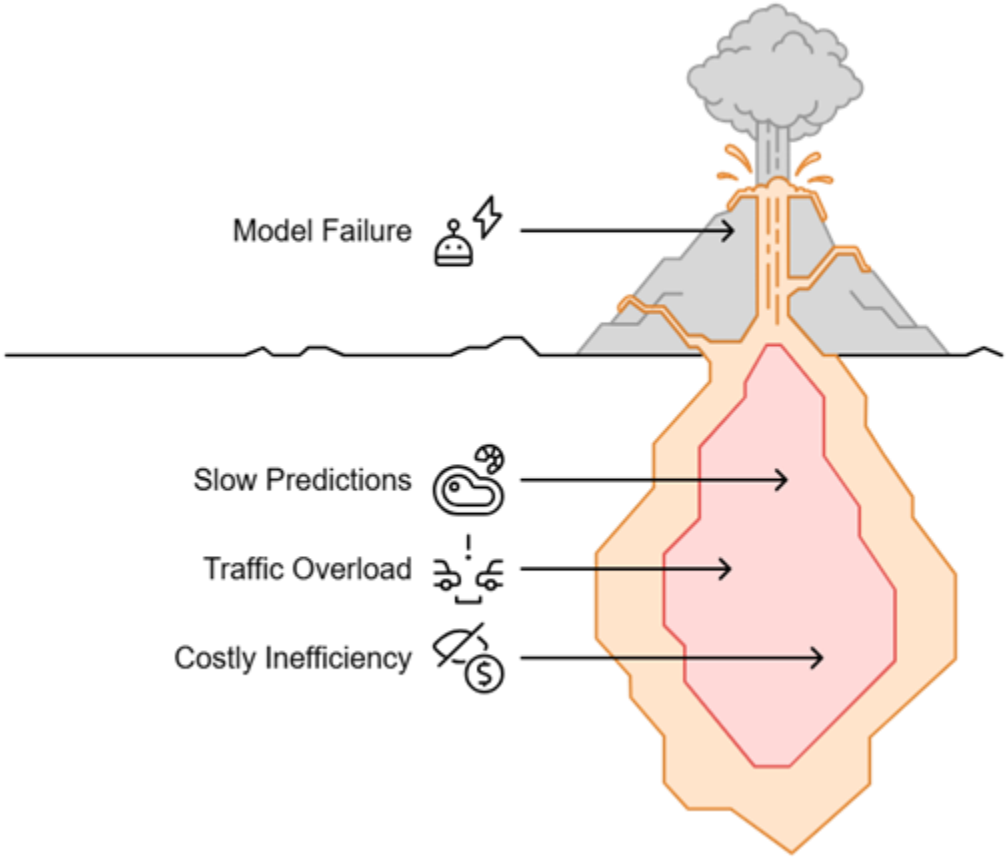
Scalability

From Prototype to Planet-Scale

- ML models aren’t just lab experiments anymore they **power:**
  - **search engines**
  - **recommenders**
  - **fraud detection**
  - **LLMs like ChatGPT**

- But what works on a laptop **doesn’t work at scale**

The Challenge



ML Scalability

Personalized AI

AI personalizes billions of user experiences.

Fast Systems

Systems respond to user requests in milliseconds.

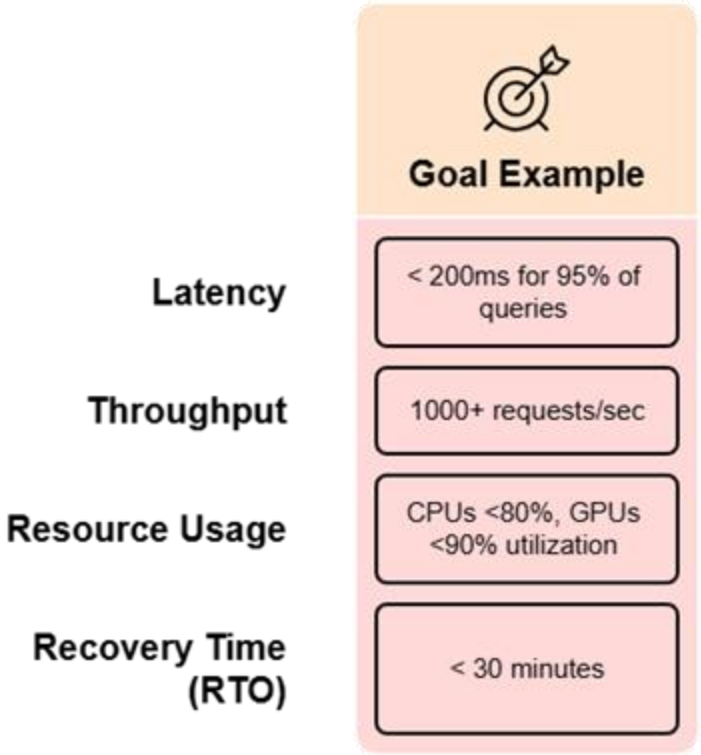
Smart Infrastructure

Infrastructure scales smartly and cost-effectively for growth.

ML Scalability

- Ability to handle **increasing data volumes, users, and model complexity** efficiently
- Involves scaling:
  - Data processing
  - Model training
  - Inference serving
  - Resource allocation

Key Scalability Metrics

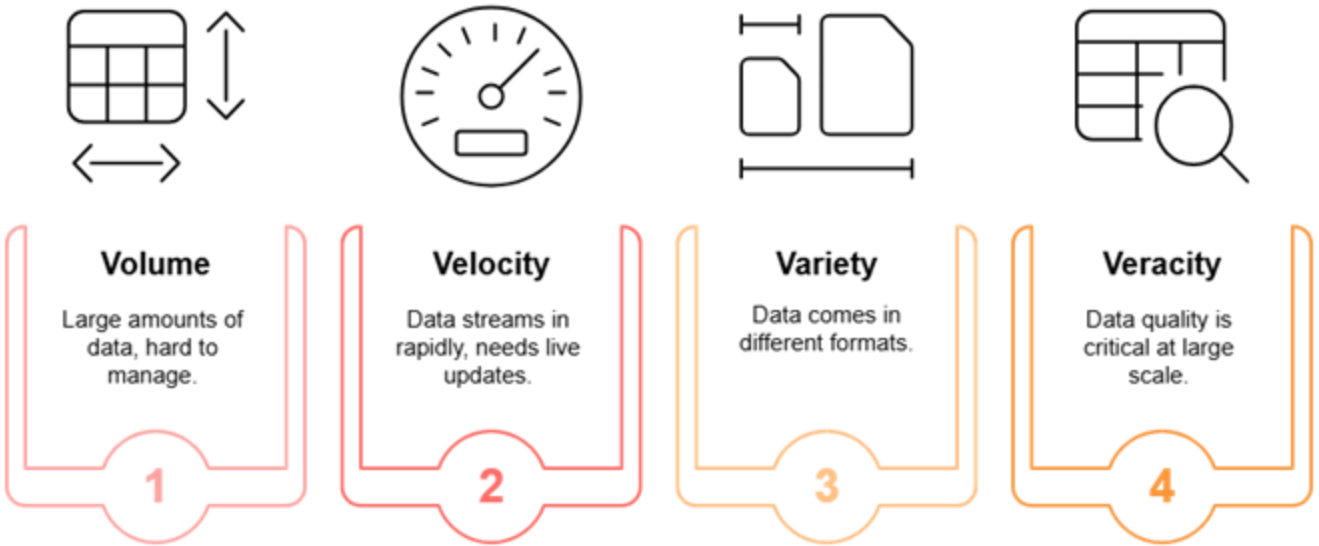


Scalability Challenges

Feature Complexity

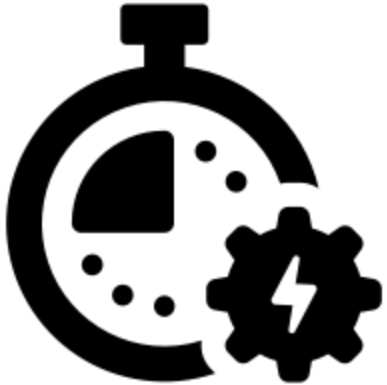
- Features come from **diverse sources**: tables, text, images
- At scale, billions of records must be **processed** and **stored** efficiently
- Feature stability over time is critical — **drift** leads to **model degradation**

Big data Challenge



High-Throughput Inference

- ML systems must serve **thousands to millions** of requests per second
- Users expect **instant responses** — delays lead to poor UX and drop-offs
- **Heavy Models = Heavy Load**



Multi-Model Serving

Model Explosion at Scale

Limited Memory & Compute

Latency Bottlenecks Appear

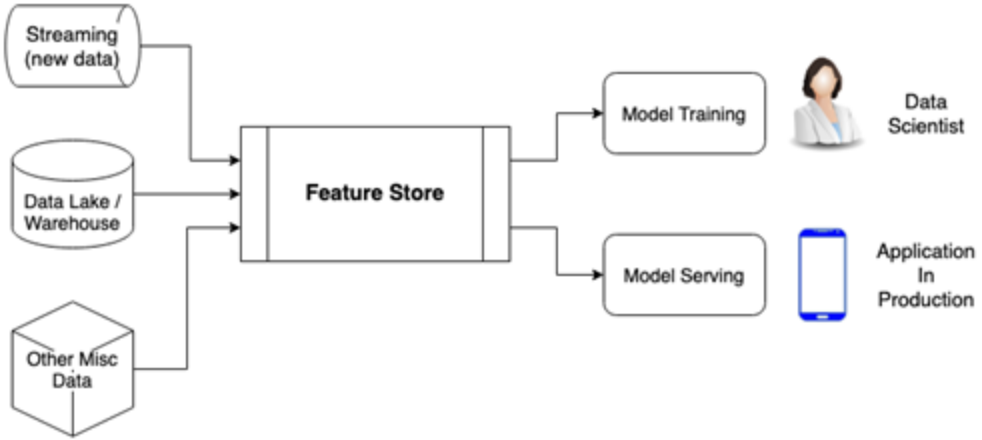
Resource Management

- **Expensive GPU/TPU** cycles are wasted with poor scheduling
- **Over-provisioning** is costly while **under-provisioning** causes latency and failures
- **Balancing** fast responses with efficient bulk processing is **tough**

Strategies for Scaling ML Systems

Scalable Data Infrastructure

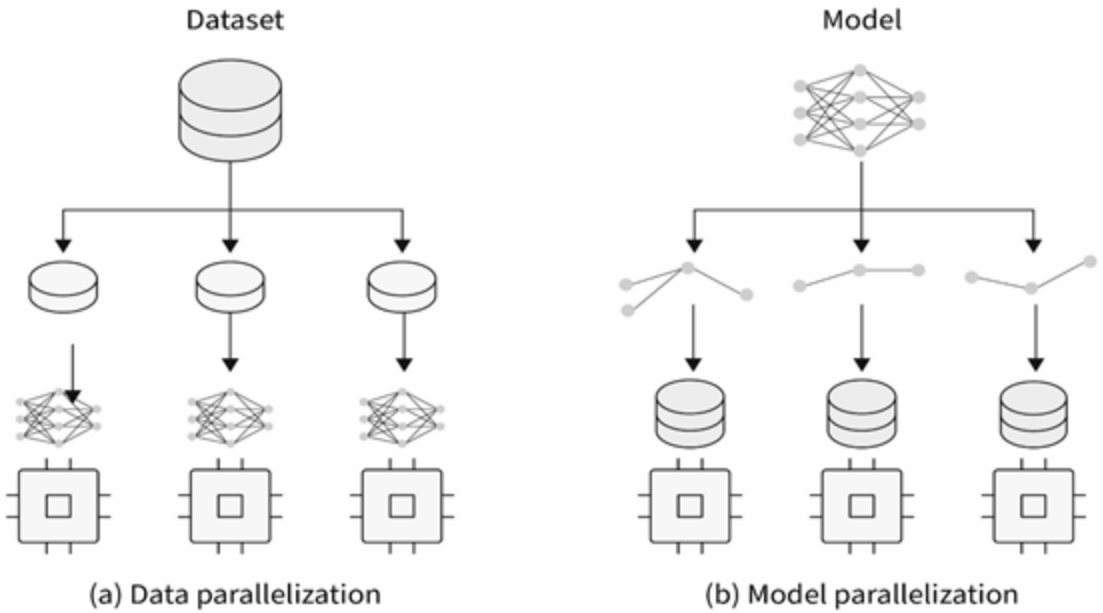
- **Feature Stores** (e.g., Feast, Redis) for feature versioning, monitoring, and reuse
- **Automated Feature Monitoring:** Detect drift and update models without manual intervention



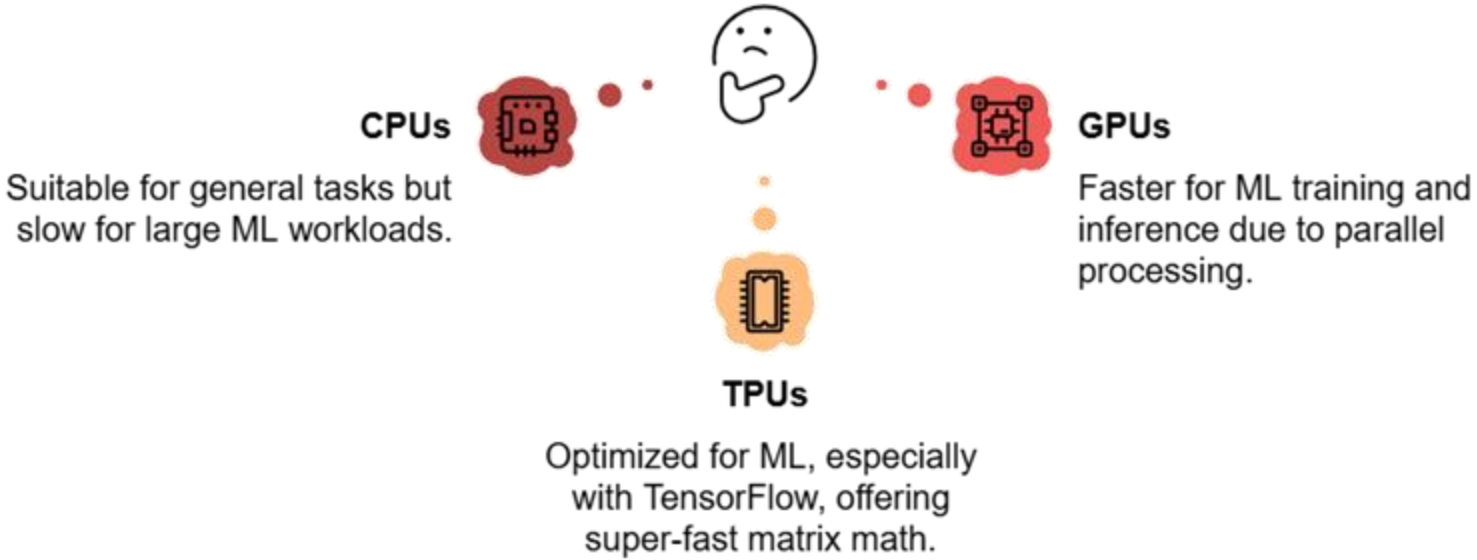
Scalable Data Infrastructure

- Use **distributed storage** (HDFS, S3, Delta Lake)
- Implement **streaming platforms** (Apache Kafka, Apache Flink) for real-time data ingestion

Parallelism

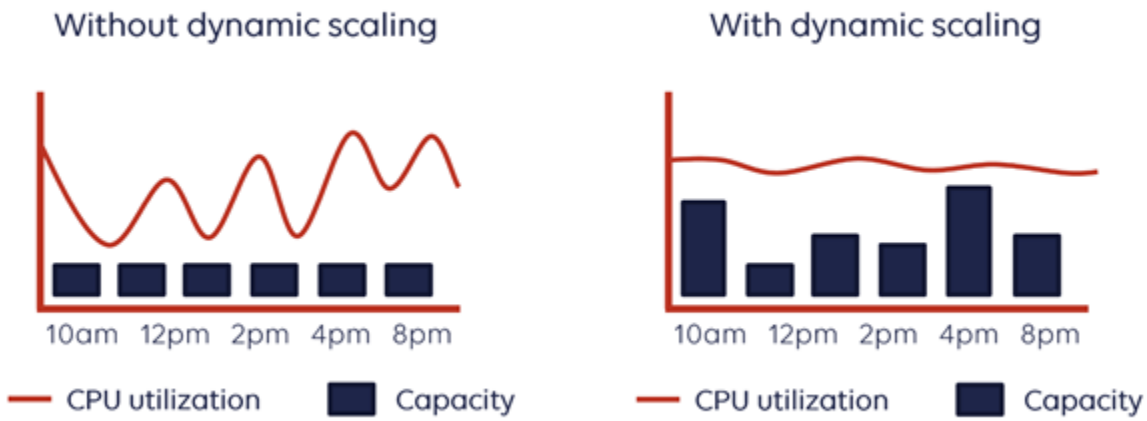


Hardware Matters

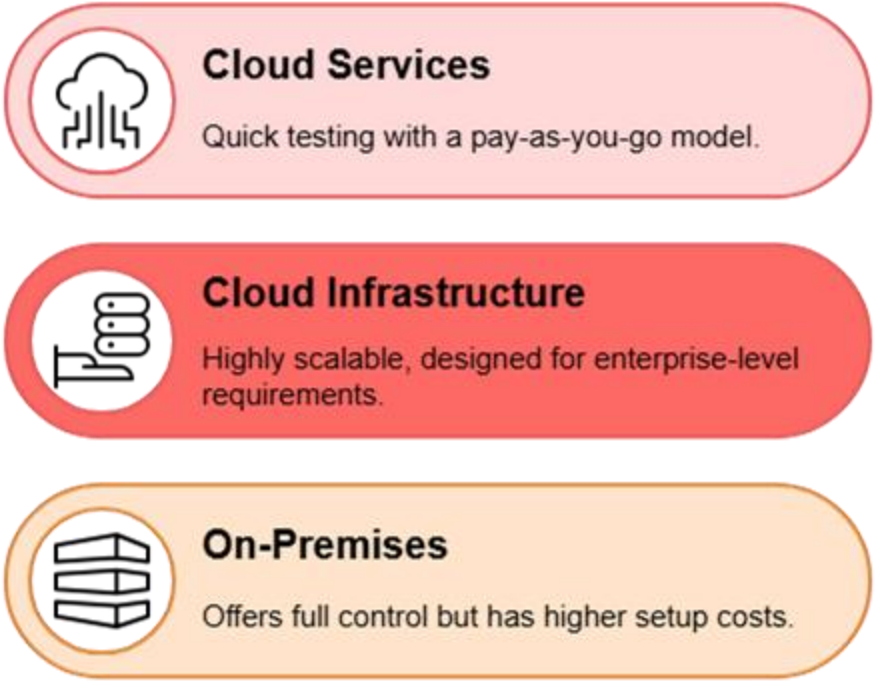


AutoScaling

- When demand **spikes** , autoscaling dynamically adds more compute
- When load drops, it **releases resources** to save cost
- Helps ensure **low latency** and **system stability** without over-provisioning

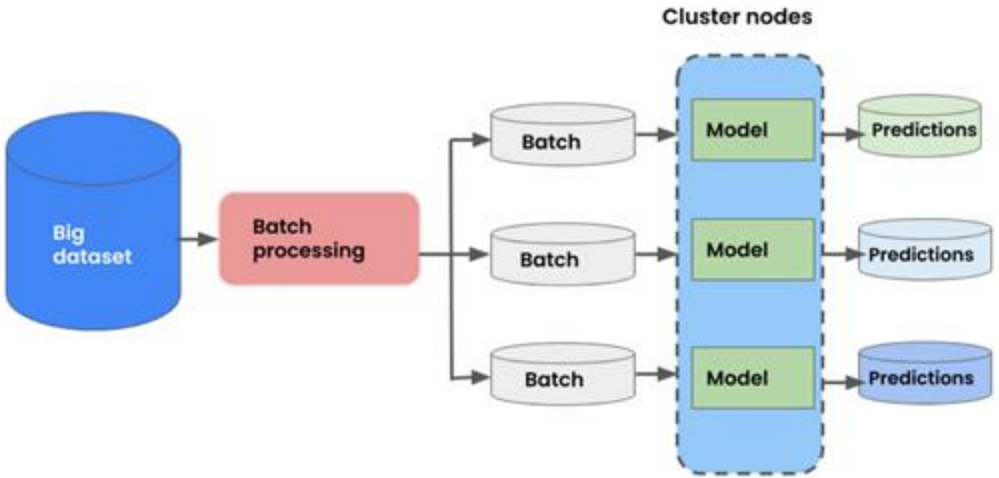


Infrastructure Options



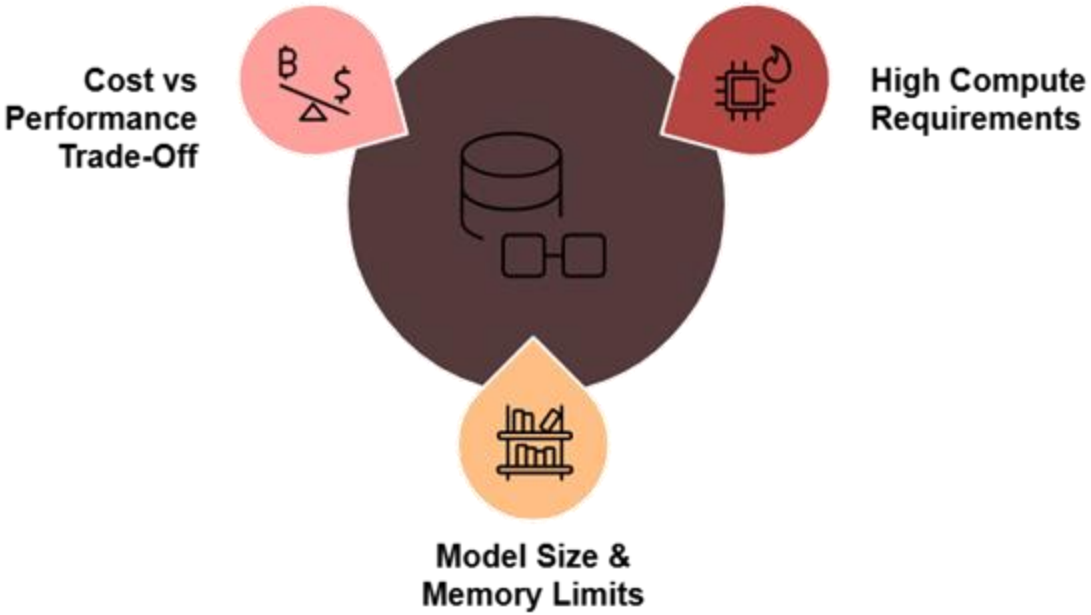
Model Batching

- **Grouping** multiple inference requests and **processing** them together
- **Reducing** per-request **overhead**
- Delivering **faster** responses



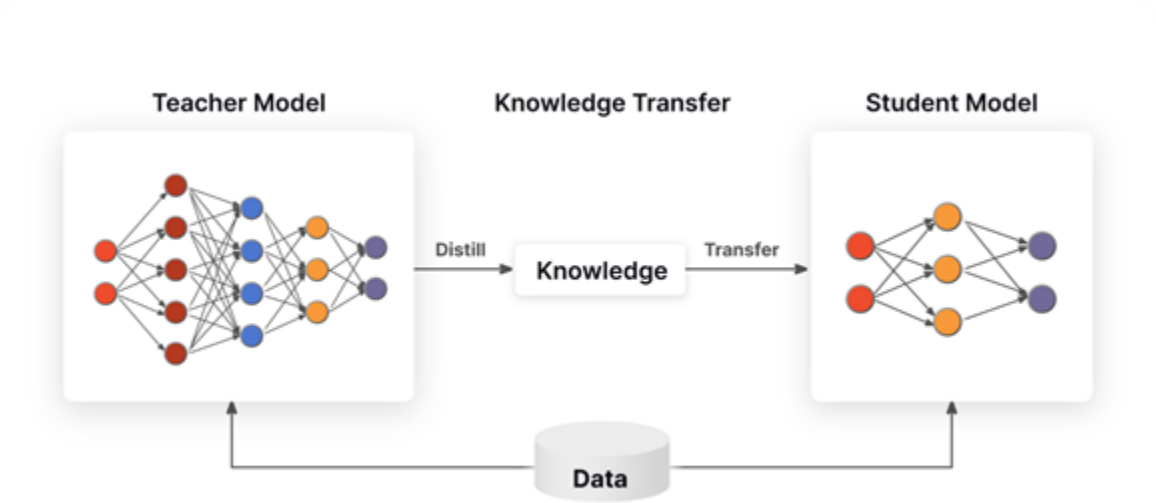
LLM at Scale

LLM-Specific Challenges



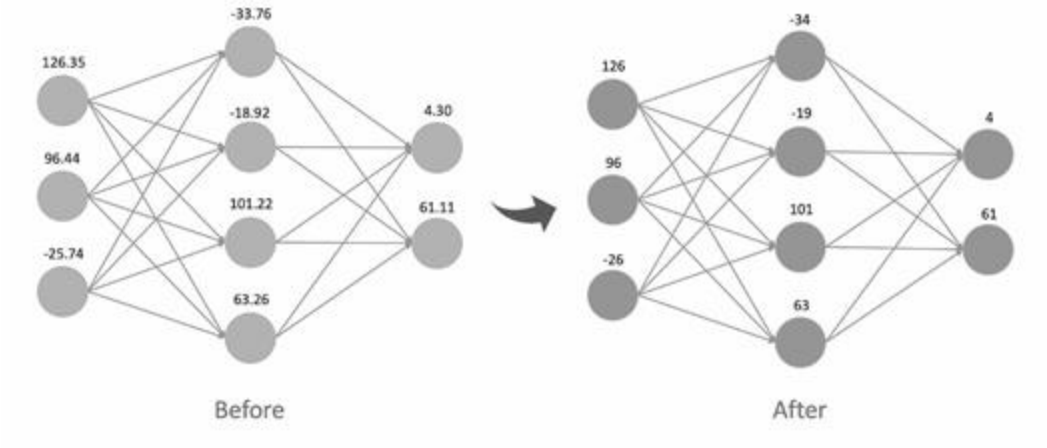
Model Distillation

- **Teacher**: Large, accurate model
- **Student**: Smaller, faster model
- Student model **mimics** the teacher's outputs with **less complexity**



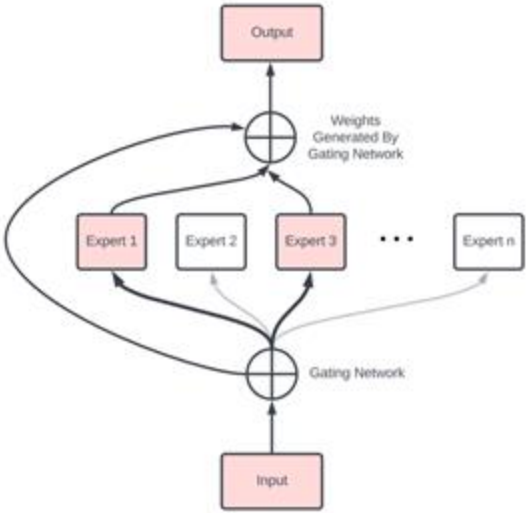
Model Quantization

- Reduces the **precision** of the model's weights
- **Decrease** size and **Improve** speed



Mixture-of-Experts (MoE)

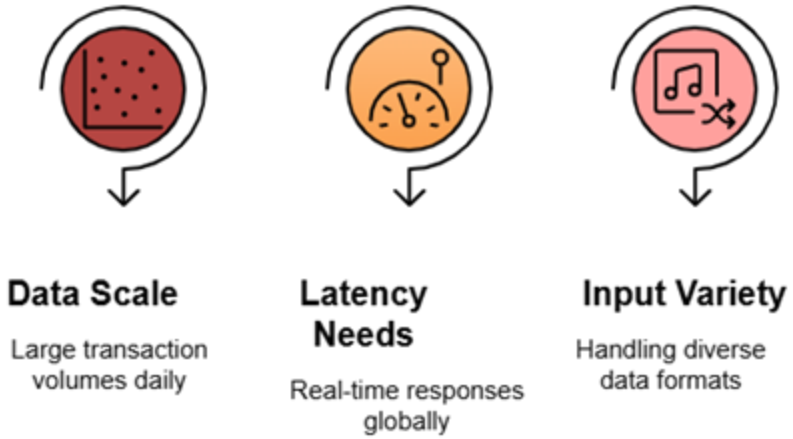
- Only a few "experts" (sub-networks) are activated per input
- A **gating mechanism** decides which experts to use based on input
- **Example:** a question about math might activate experts trained more on numerical reasoning



Case Study: Supermarket's GenAI Platform

Challenges

- **Objective:**
  - Scale generative AI to enhance customer experience across e-commerce, inventory, and operations
- **Challenges:**



Solutions

- **Solution:**
  - Built a centralized LLM platform
  - Used Mixture of Experts (MoE) and model distillation
  - Adopted hybrid cloud deployment strategy

- **Tech Stack:**
  - Fine-tuned open-source models (e.g., Falcon, LLaMA)
  - Kubernetes-based orchestration for autoscaling Hardware: NVIDIA A100 clusters and TPUs

Impact

Multilingual Support

Supported multilingual capabilities

Inference Costs

Reduced model inference costs

Real-time Personalization

Enabled real-time personalization

Summary

Key Takeaways

- ML systems must scale across **data, models, inference, and infrastructure**
- **Feature complexity** and **big data** create storage and processing challenges
- **Efficient resource management** (GPU/TPU, autoscaling) is vital for cost and performance
- **Scalability solutions** include parallelism, model compression, and smart infrastructure
- **LLMs require specialized strategies**, not just more hardware

Break


O'REILLY®

O'REILLY

Advanced Kubernetes for ML Deployments

Ammar Mohanna

Learning Objectives

By the end of this lesson, you will be able to:

- Understand the **limitations** of basic Kubernetes setups for real-world ML applications
- Explore **advanced Kubernetes features** including:
  - High availability strategies
  - Auto-scaling mechanisms (HPA, VPA, Cluster Autoscaler)
  - Resource quotas and fair sharing
- Apply these features to make **ML deployments** scalable, resilient, and production-ready

Kubernetes

Recap – Kubernetes for ML Deployments

- **Pods**: The smallest deployable units of compute
- **Deployments**: Manage pod replicas, updates, and rollbacks
- **Services**: Stable endpoints that expose your applications

But is that enough for real-world traffic?

The Real Challenge Begins

Cost Optimization

Reducing expenses while maintaining performance

Resource Sharing

Distributing resources equitably among teams

Scalability Needs

Handling sudden traffic increases efficiently

Real-time Predictions

Ensuring predictions happen instantaneously

Kubernetes Advanced Features

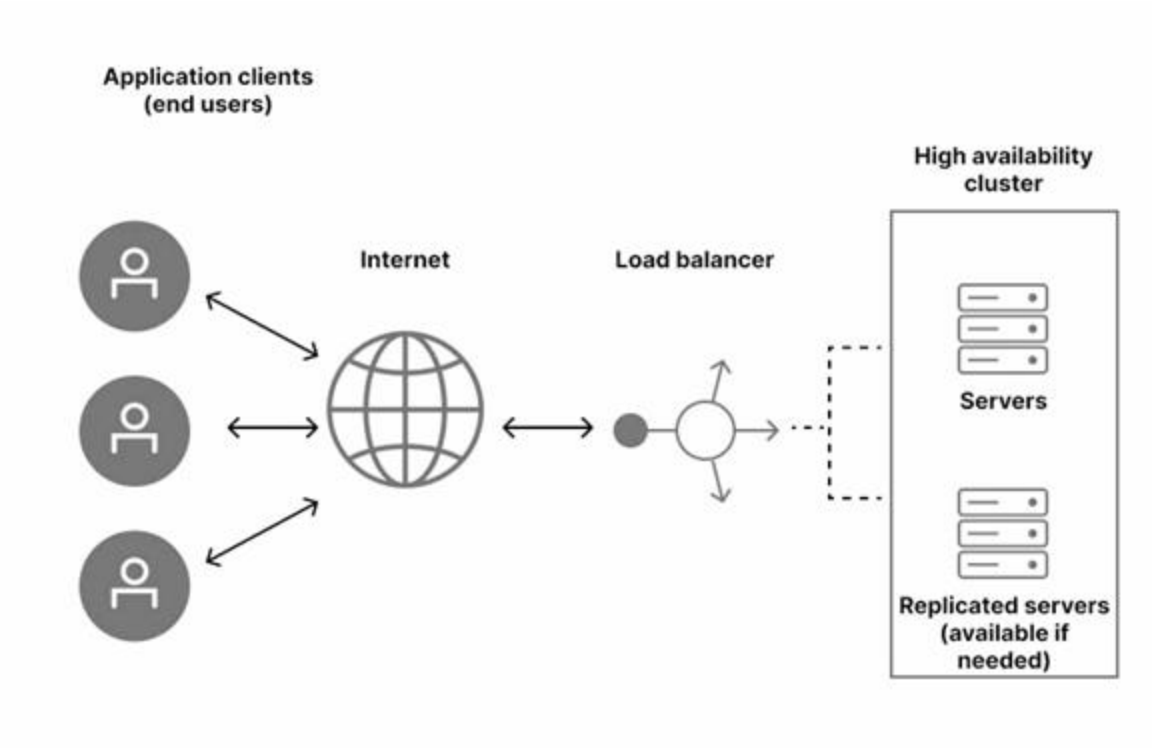
Kubernetes offers different advanced features to:

- **Scale** Intelligently
- Stay **Resilient**
- Operate **Efficiently** under real-world production demands

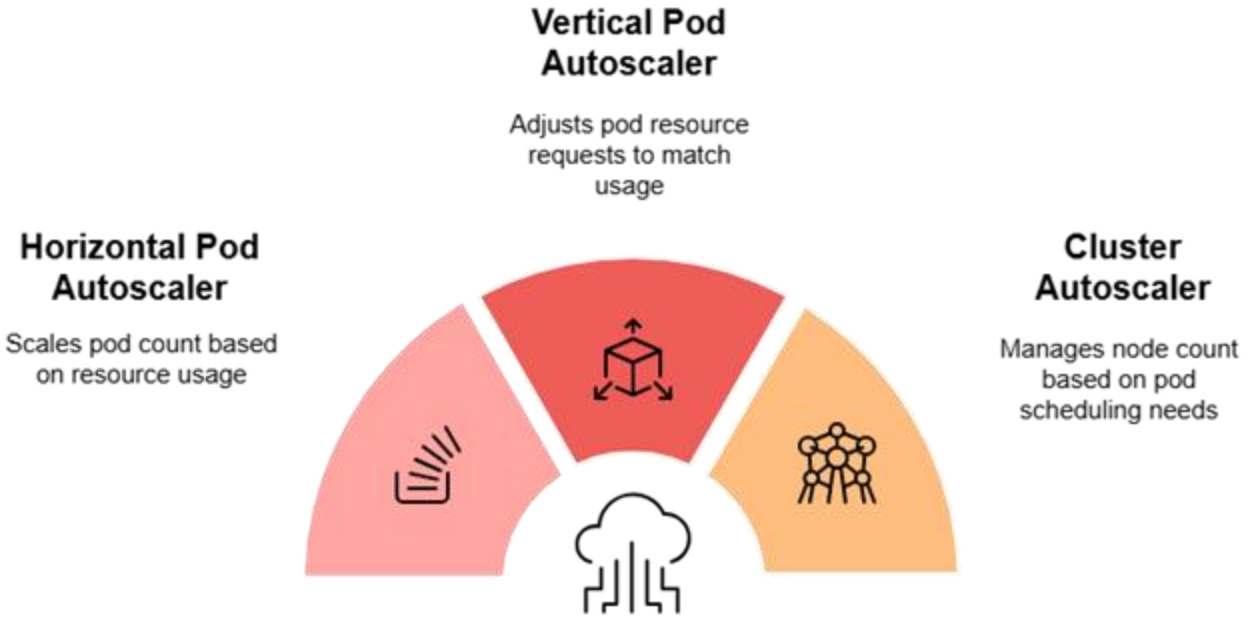
Kubernetes Features

High Availability for ML Services

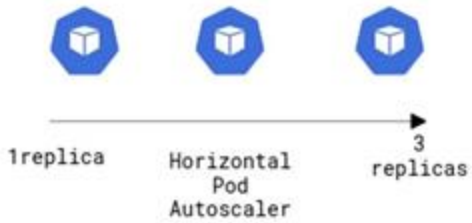
High Availability for ML Services: Load Balancing

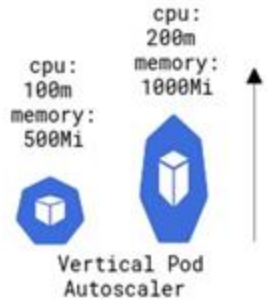


Advanced Auto-Scaling Patterns



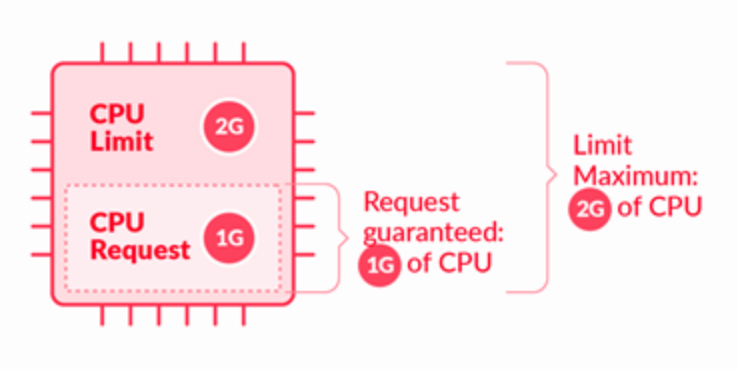
Horizontal vs. Vertical Auto-Scaling



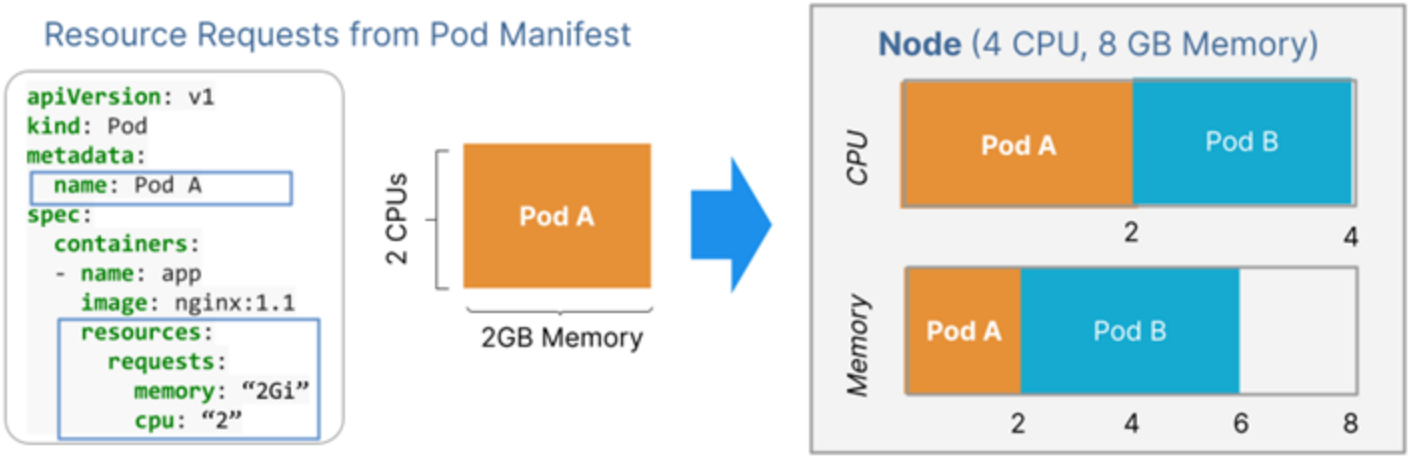


Resource Quotas and Limits

- Prevents any one team or app from **consuming** all the cluster **resources**
- Set **max total** CPU, memory, pods, etc., per namespace
- Enforce **fair resource sharing** across teams/projects
- **ML Example:** Ensure a GPU-hungry training job doesn't starve real-time inference services

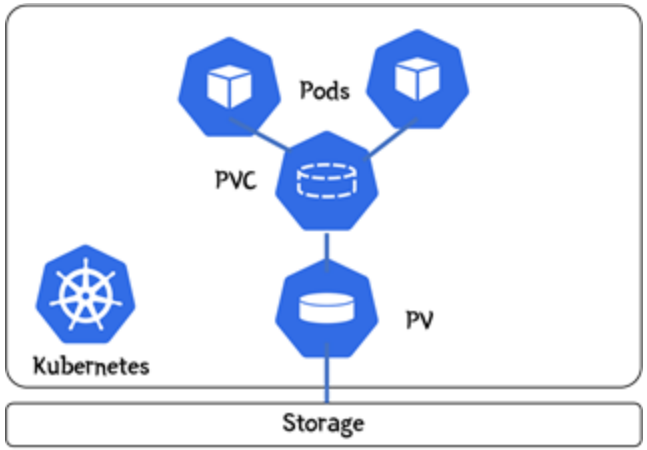


Resource Quotas and Limits



Storage in Kubernetes

- **Persistent Volumes (PVs):**
  - They exist independently of your pods
  - Stay intact even after a pod is deleted
- **Persistent Volume Claims (PVCs)**
  - PVCs are like a "request" for storage by a pod
  - The pod doesn't care where the storage comes from — it just asks for what it needs



Security in Kubernetes

Summary

Key Takeaways

- **Kubernetes** enables **production-grade** scalability and reliability
- **High Availability & Scaling:** Load balancers, replicas, and auto-scaling adapt to demand
- **Resource Efficiency:** Quotas, limits, and persistent storage optimize usage and reliability
- **Security:** RBAC, namespaces, and secrets safeguard access and data

Hands-on: HPA in Kubernetes


O'REILLY®

O'REILLY

ML Workflow Orchestration

Ammar Mohanna

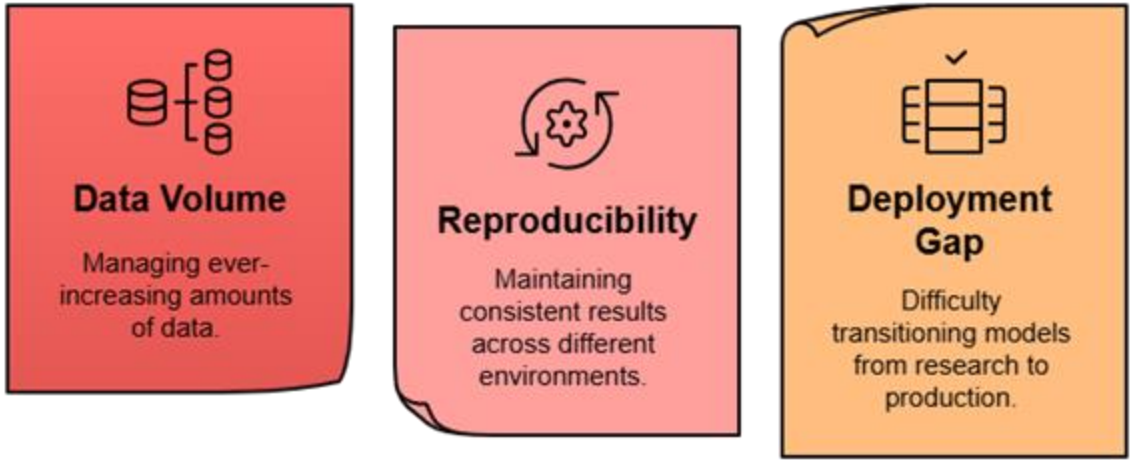
Learning Objectives

By the end of this lesson, you will be able to:

- Understand **what orchestration** means in the context of ML workflows
- Recognize why **orchestration** is essential for building scalable and reliable **ML systems**
- Explain the **core components** of **Kubeflow** and how they integrate with Kubernetes
- Understand the role of Kubeflow in **LLMOps** tasks

Orchestration

The Modern ML Landscape



What Is Orchestration?

- **Orchestration** = defining, scheduling, managing pipeline steps
- Ensuring that different **steps** of a process **work together** seamlessly
- Defining a **workflow**

Why Orchestrate?

Script Elimination

Reduces need for manual scripting

Correct Ordering

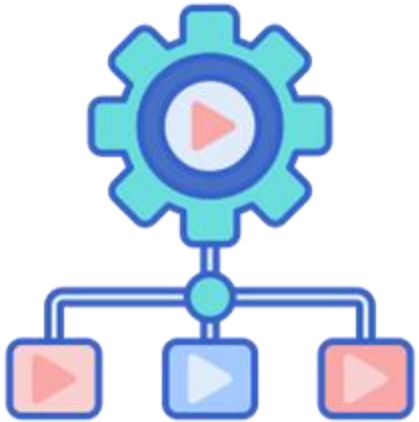
Ensures tasks run in sequence

Failure Management

Handles retries and recovery from errors

Orchestration in Machine Learning

- Ensures tasks happen in the **correct sequence** (e.g., preprocessing before training)
- Manages **dependencies** between tasks to avoid error

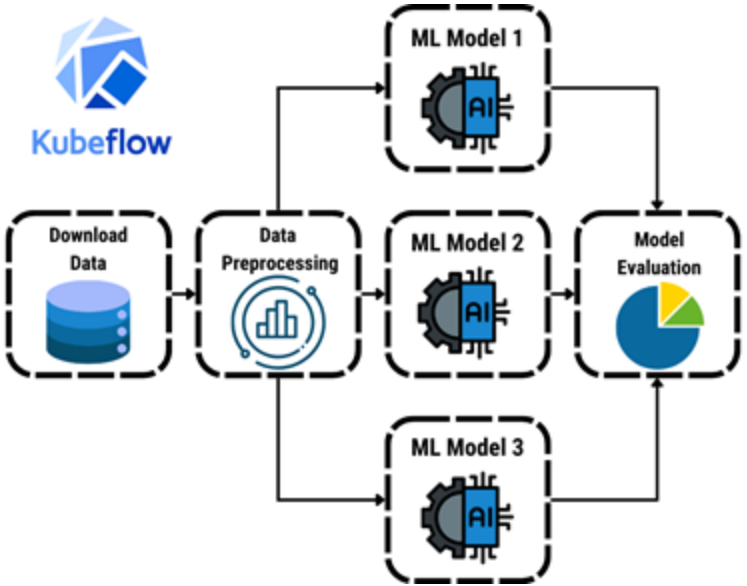


Orchestration Tools for Machine Learning

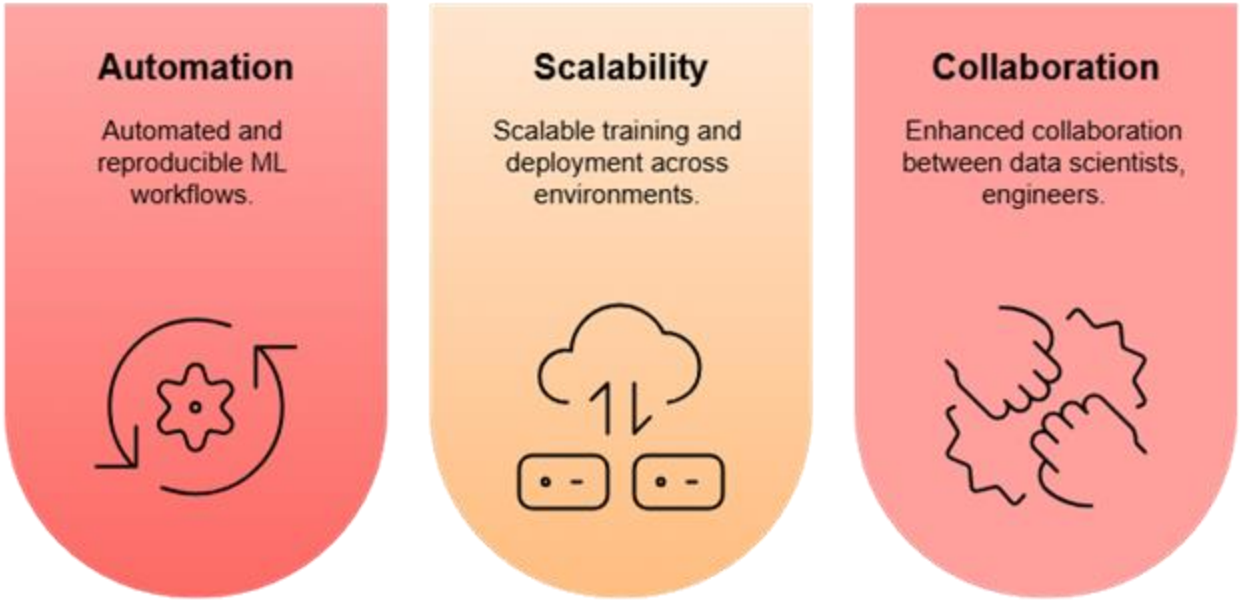
Kubeflow

Introduction to Kubeflow

- Open-source platform that serves as an **orchestrator** for **machine learning (ML) workflows**
- Simplifies the **automation** of tasks across the entire ML lifecycle
- Originally developed by **Google**



Benefits

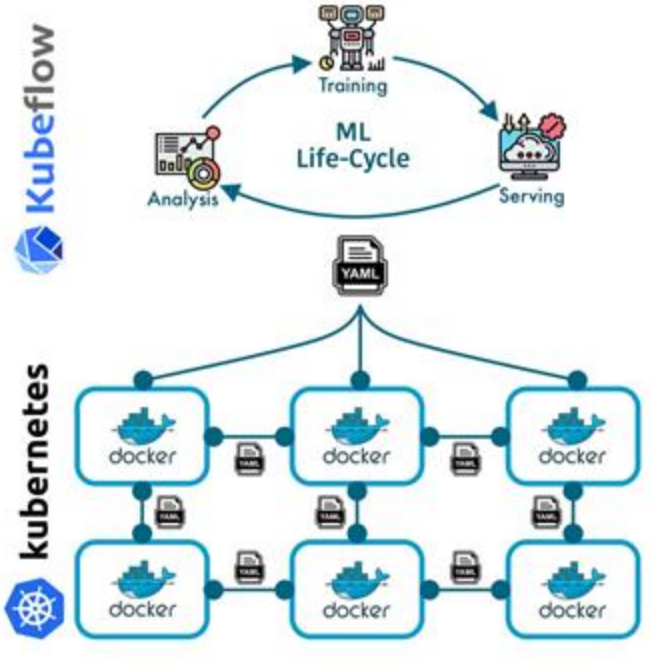

Automation
Automated and reproducible ML workflows.
Scalability
Scalable training and deployment across environments.
Collaboration
Enhanced collaboration between data scientists, engineers.

Kubeflow Integration

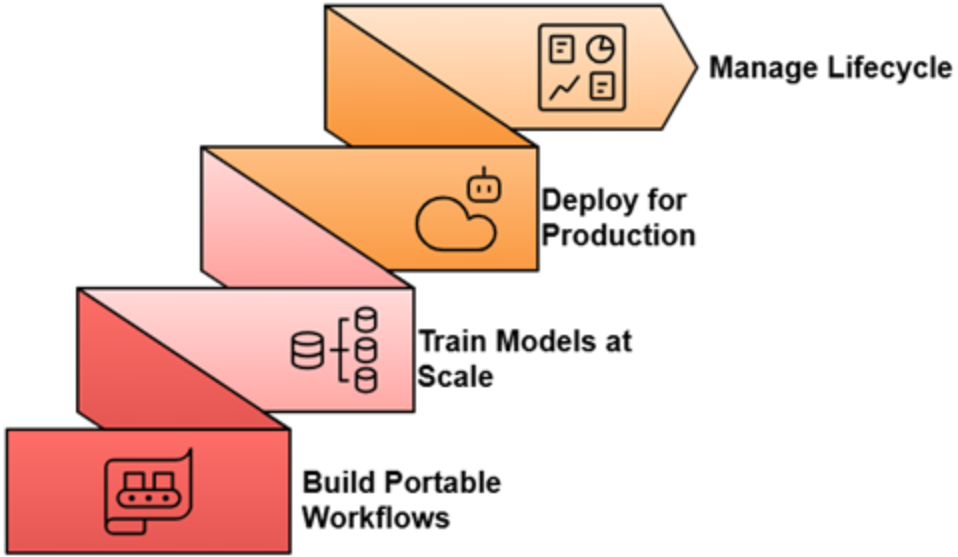
- Seamlessly integrates with **Kubernetes** to handle orchestration
- **Runs Anywhere**
  - Google Cloud Platform (GCP), Amazon Web Services (AWS), Azure, IBM Cloud — and even on-premises

Kubeflow and Kubernetes

- **Kubeflow** acts as the control center for machine learning workflows
- All tasks in Kubeflow are **declared in YAML files**
- **Kubernetes** runs the actual workloads, leveraging Docker containers

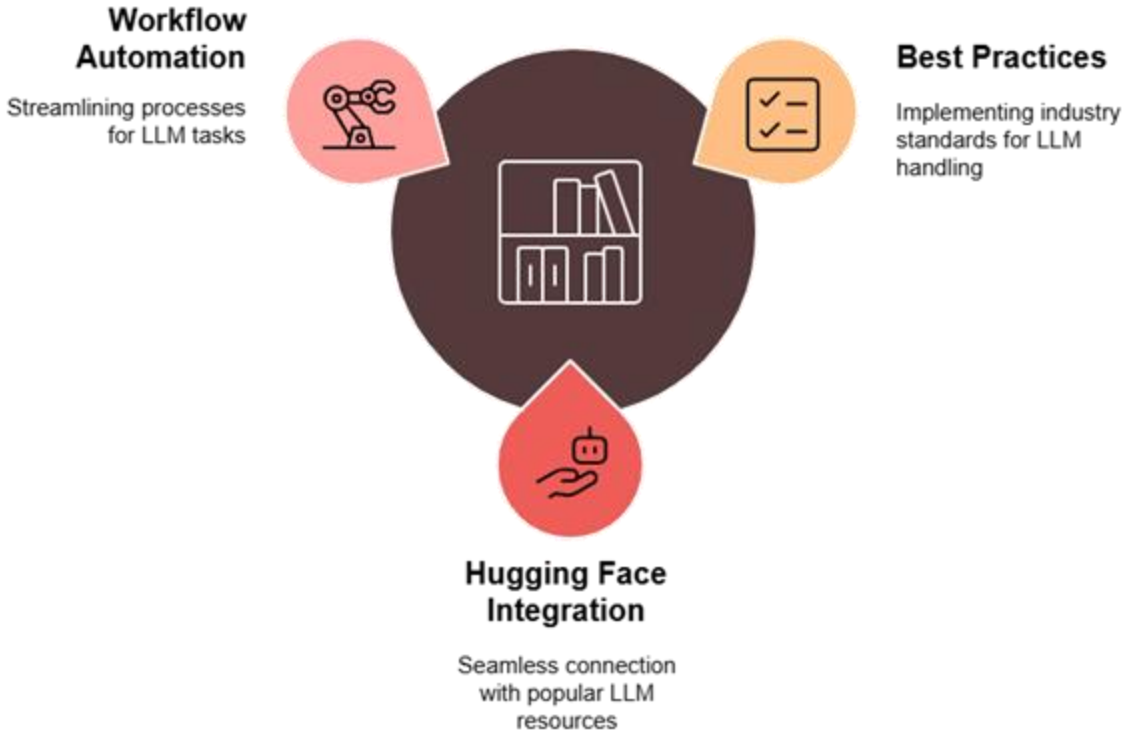


Use Cases of Kubeflow

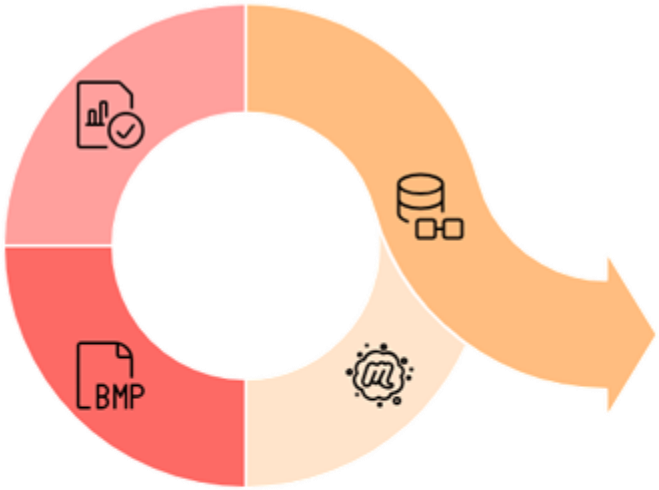


LLMOps with Kubeflow

Kubeflow Pipelines for LLMOps



Example: Automating Prompt Engineering



1

Generate Prompts

Creates prompt variations for LLM input

2

LLM Inference

Sends prompts to LLMs for outputs

3

Evaluate Outputs

Compares outputs using metrics

4

Log Results

Stores triplets for analysis

Summary

Key Takeaways

- **ML orchestration** automates steps in the machine learning workflow
- **Kubeflow** is an **open-source platform** for end-to-end ML orchestration
- Tools like **Notebooks, Pipelines, Katib,** and **KServe** streamline ML tasks
- Supports **LLMOps** for automating prompt testing, fine-tuning, and evaluation


O'REILLY®

O'REILLY

Advanced Experimentation

Ammar Mohanna

Learning Objectives

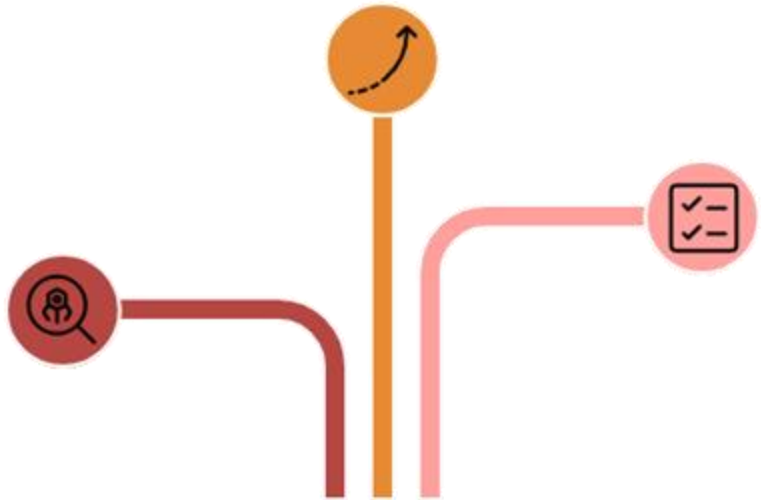
By the end of this lesson, you will be able to:

- Understand why **experimentation and validation** are critical for ML models in production
- Explain and differentiate between **A/B testing, canary release,** and **multi-armed bandit strategies**
- Make informed decisions on **which experimentation strategy** to use in real-world ML deployments

Model Experimentation & Validation

Common Failure Stories

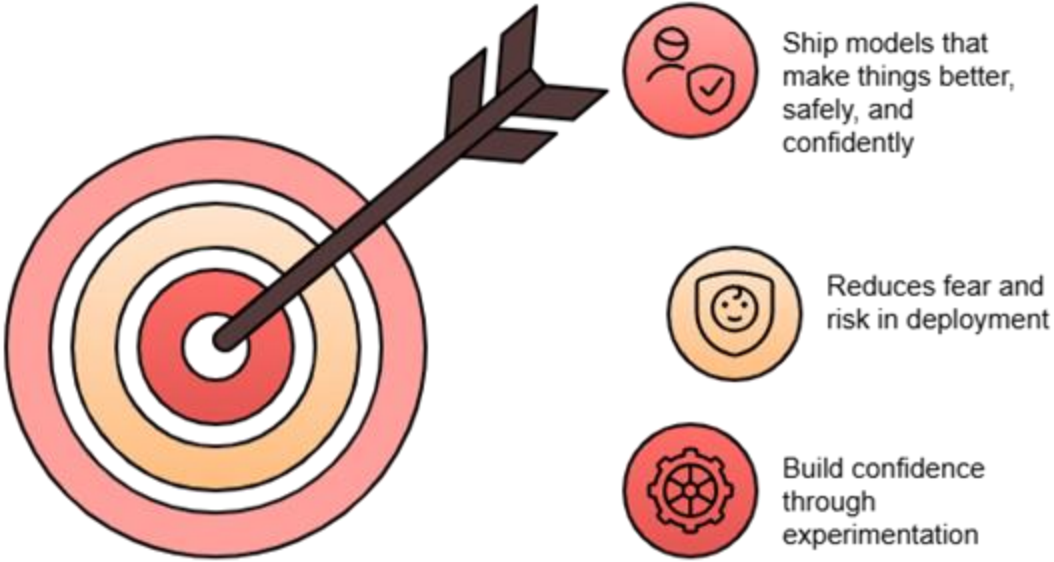
Rollouts cause outages or degraded user experience



New model tanks business metrics despite good validation

Teams spending months retraining and re-tuning, only to find no improvement

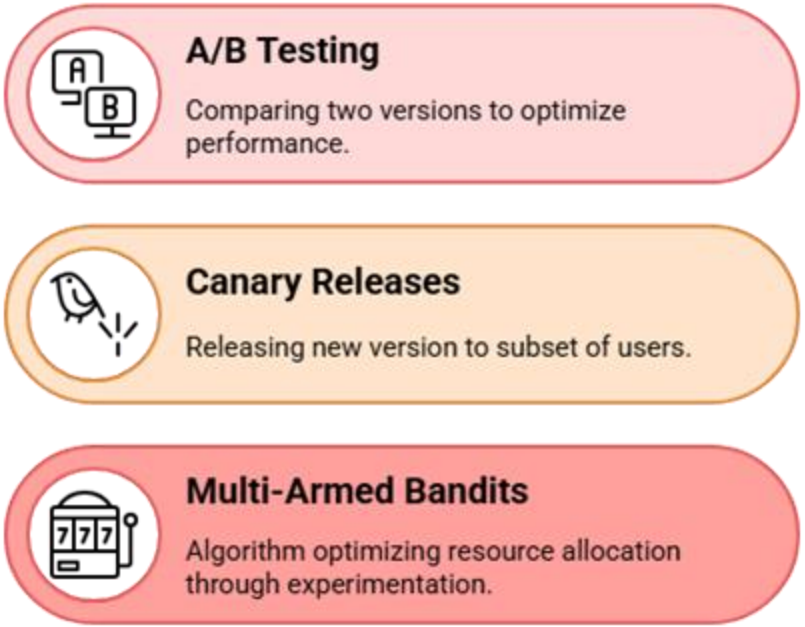
The Goal


Ship models that make things better, safely, and confidently
Reduces fear and risk in deployment
Build confidence through experimentation

Model Experimentation and Validation

- Systematically **testing models** before full deployment
- **Validating** that new models perform **better** in the real world
- **Outcome:**
  - Enables continuous **improvement** and **learning**
  - Build **trustworthy** ML systems that evolve **safely** over time

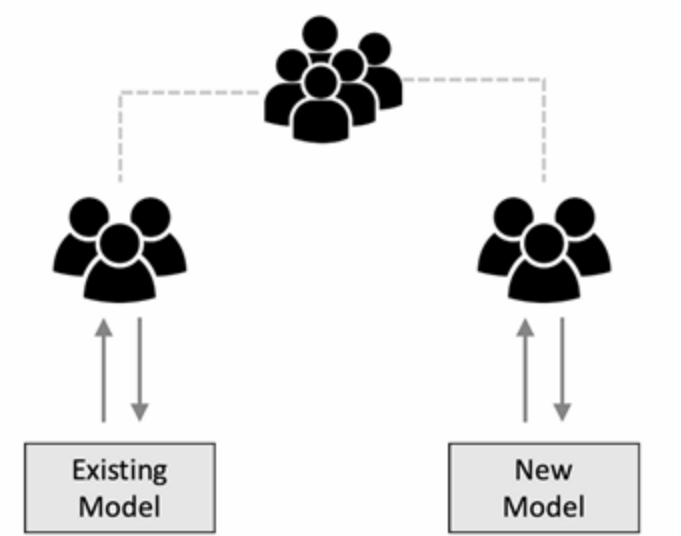
Techniques to Mitigate Deployment Risk



Model Experimentation Strategies

A/B Testing

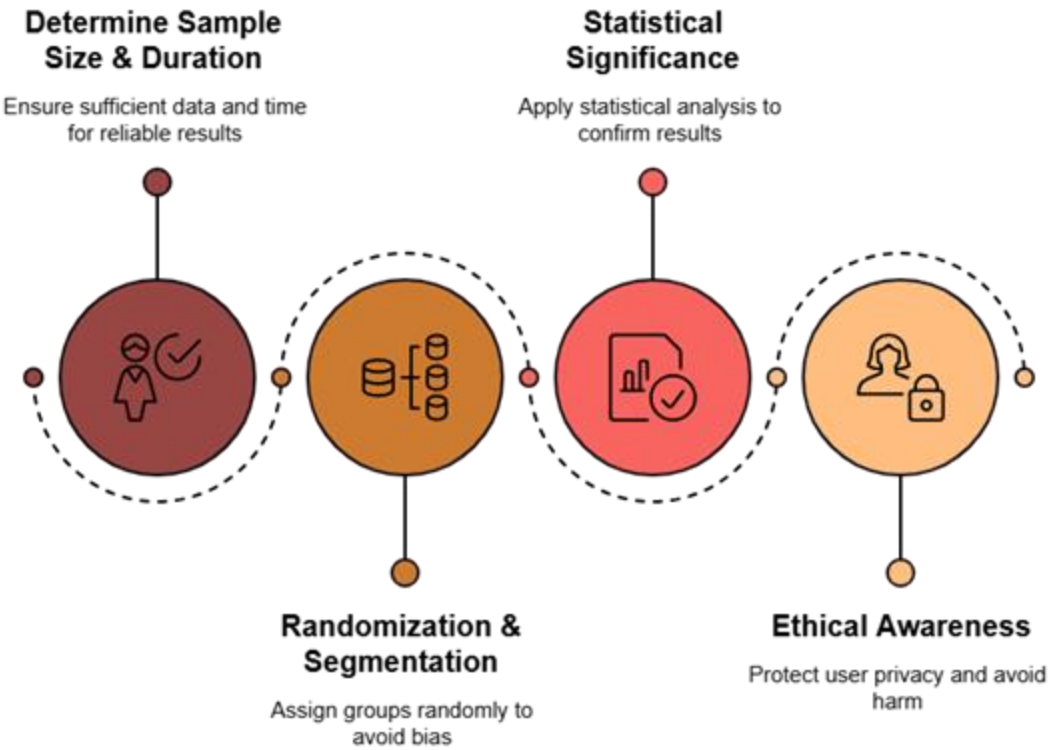
1. **Deploy** both the **current model** and **the new** candidate model
2. Route a **random split** of **traffic** (e.g., 50% to each model)
3. **Monitor** and **compare** performance based on predefined metrics (e.g., conversion rate, accuracy)



A/B Testing

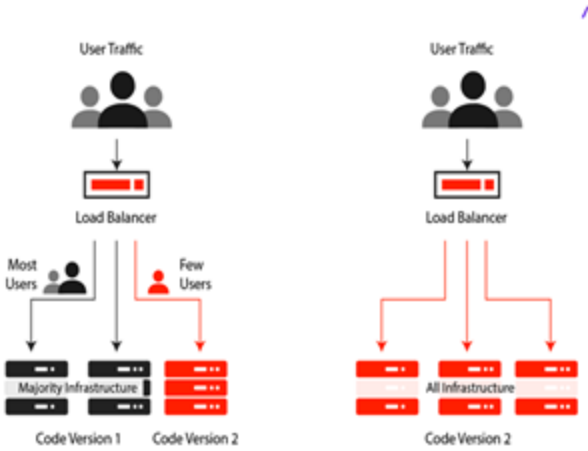

Days
Weeks
Offline Evaluation
Success
A/B Testing
Success
Deploy model to all users
Adjust
Evaluation Metrics
Real-time Feedback

Key Consideration for A/B Testing



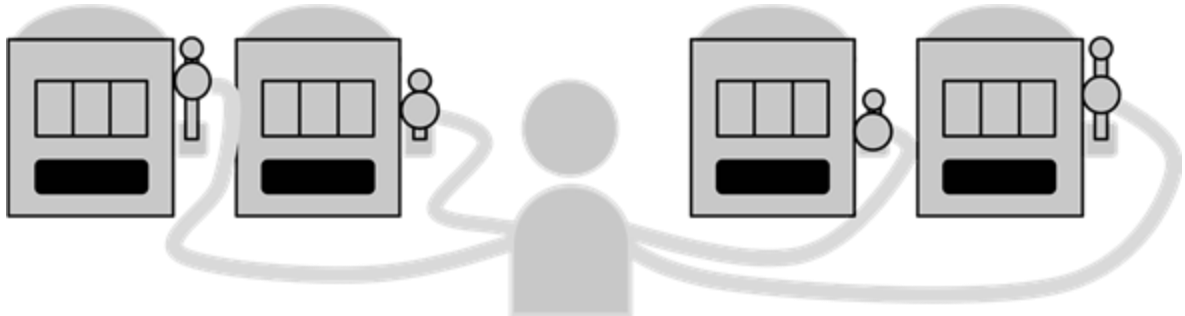
Canary Release

- **progressive rollout** technique to **minimize risk**
- **Implementation:**
  1. Deploy **canary model** (candidate model) alongside the existing one
  2. Start with a **small portion** of traffic (1-5%), **increase** if performance is good
  3. **Monitor** performance and abort if metrics degrade



Multi-armed Bandits

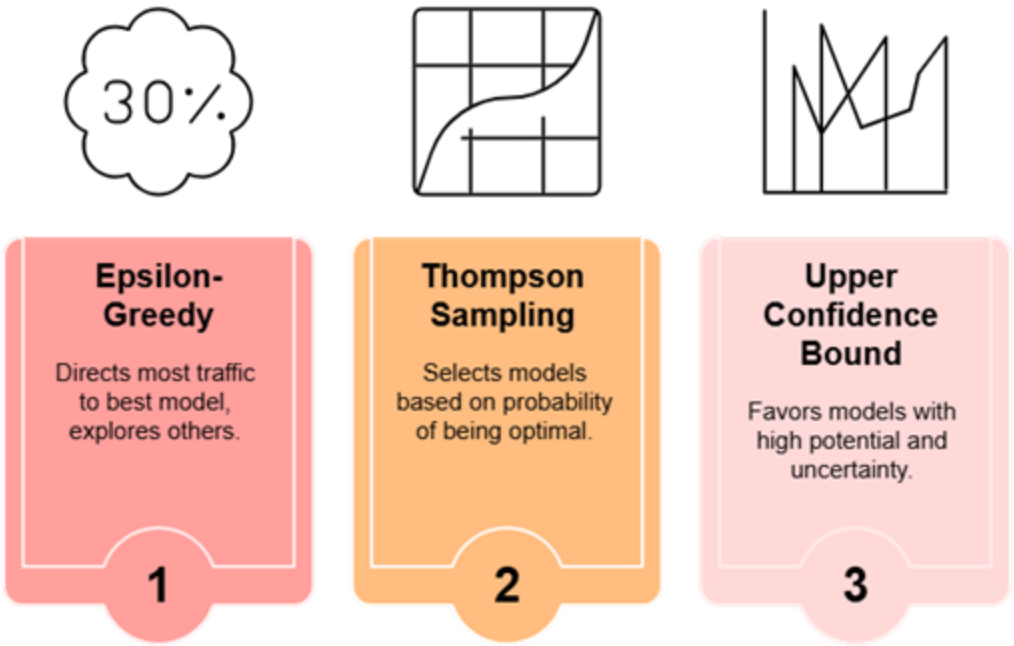
1. Each model = a **slot machine** with unknown accuracy (payout)
2. Initially, **distribute traffic** evenly or randomly across models
3. **Dynamically** route traffic based on model **performance**



Bandits vs. A/B Testing

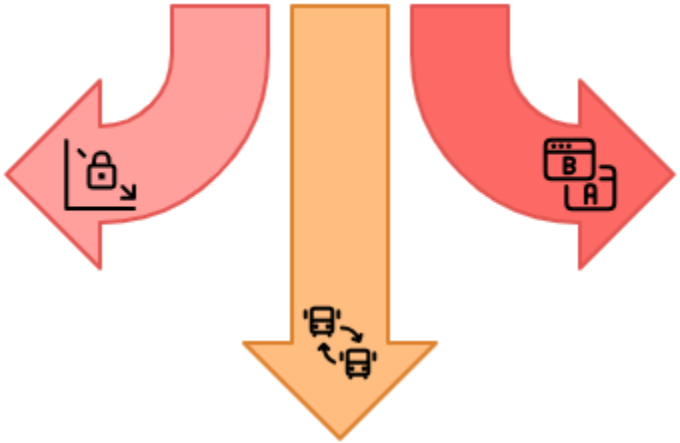
Bandits	A/B Testing
Stateful: tracks performance continuously	Stateless: random traffic assignment
Efficient: routes traffic to better models quickly	Less efficient: waits for full experiment
Needs online predictions & fast feedback loops	Works with batch or online predictions
Harder to implement	Simpler to implement

Bandit Algorithms



Best Practices

When to Use each



Canary Release

When you need to gradually increase traffic and a safe rollback if performance drops

A/B Testing

Best when we can collect metrics before deciding

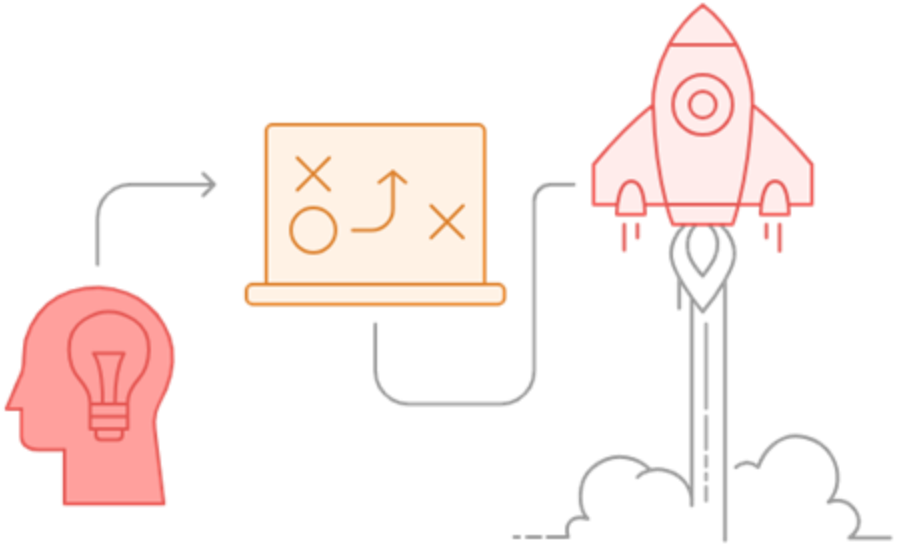
Bandit Algorithms

Best when fast and continuous feedback is available

Choosing the Right Strategy

- Deploy fraud detection model on 5% transactions, gradually increase if stable → **Canary Release**
- Test new vs. current recommendation model; split users 50/50 → **A/B Testing**
- Dynamically send more traffic to better ad ranking models based on clicks → **Bandit Algorithms**

Best Practices



Clear Pipelines

A good evaluation process = clear pipelines + automation

Define Tests

Define which tests to run, in what order, and pass/fail thresholds

Automate Pipelines

Aim for automated pipelines integrated into CI/CD for ML

Summary

Key Takeaways

- **Model experimentation & validation** protects against offline vs. online performance gaps
- **A/B Testing** compares models using randomized traffic splits
- **Canary Releases** gradually roll out new models to reduce risk
- **Multi-Armed Bandits** dynamically shift traffic to better models over time

Hands-on: A/B Testing for Diabetes Prediction

Break


O'REILLY®

O'REILLY

Operational Challenges for LLMs

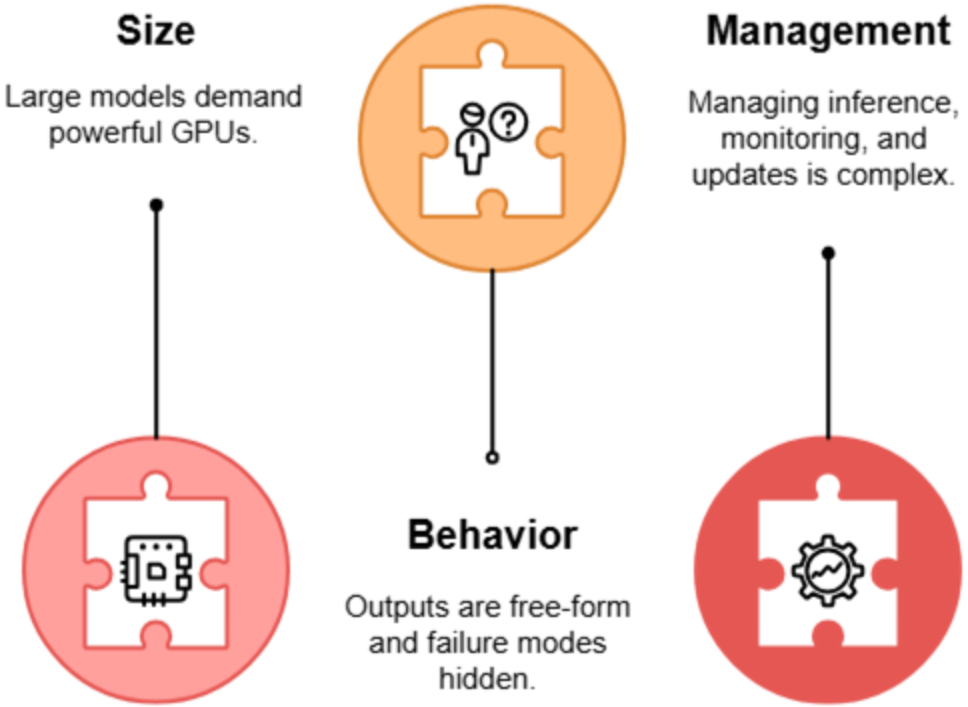
Ammar Mohanna

Learning Objectives

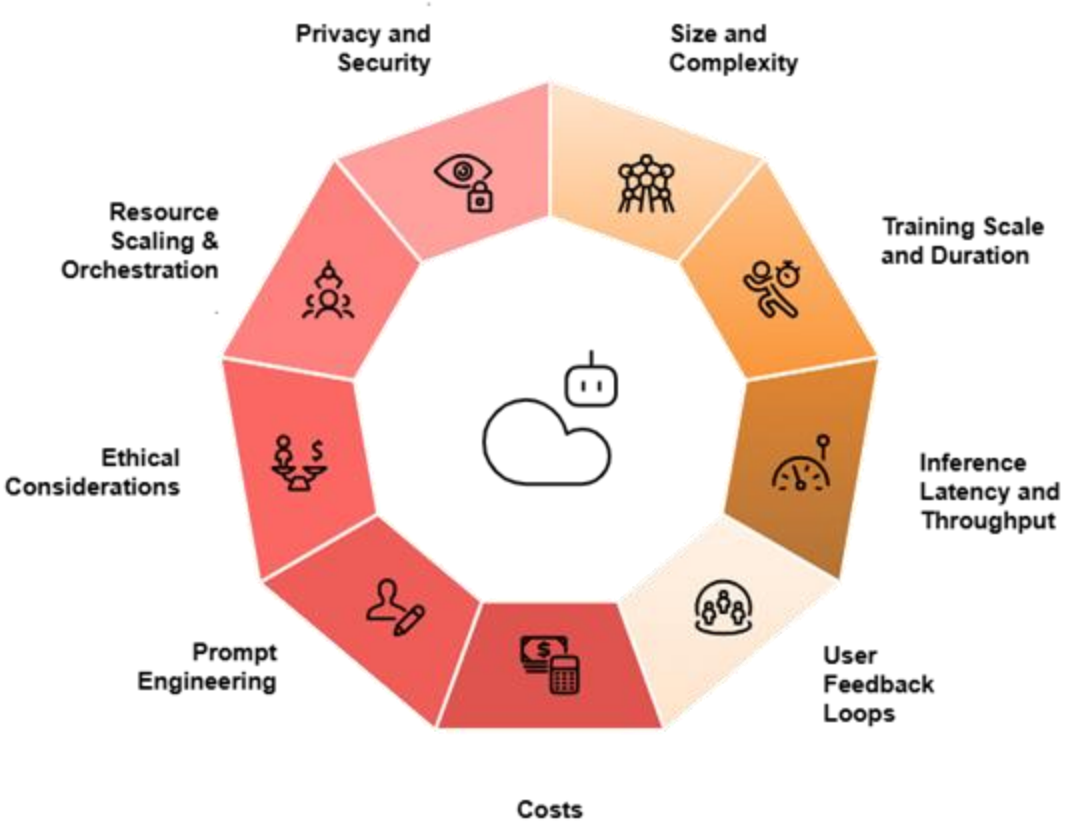
By the end of this lesson, you will be able to:

- Identify key **operational challenges** in deploying LLMs
- Explain **causes** of latency, scaling, and cost issues
- Apply practical **solutions** for performance and feedback

Why Are LLMs Operationally Challenging?



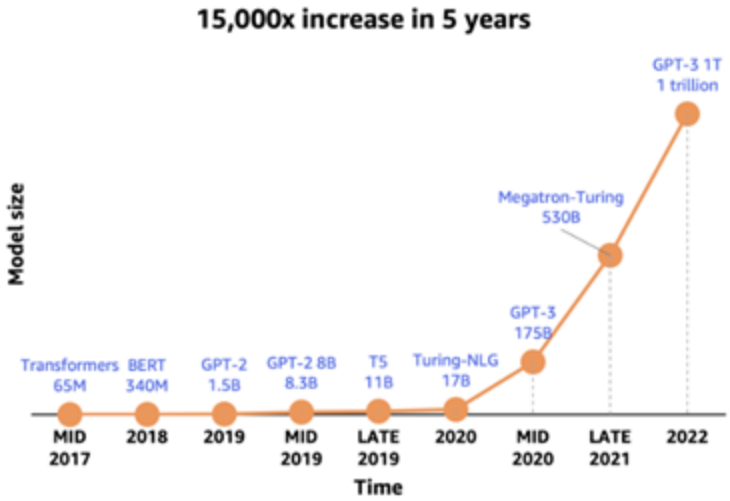
Operational Challenges



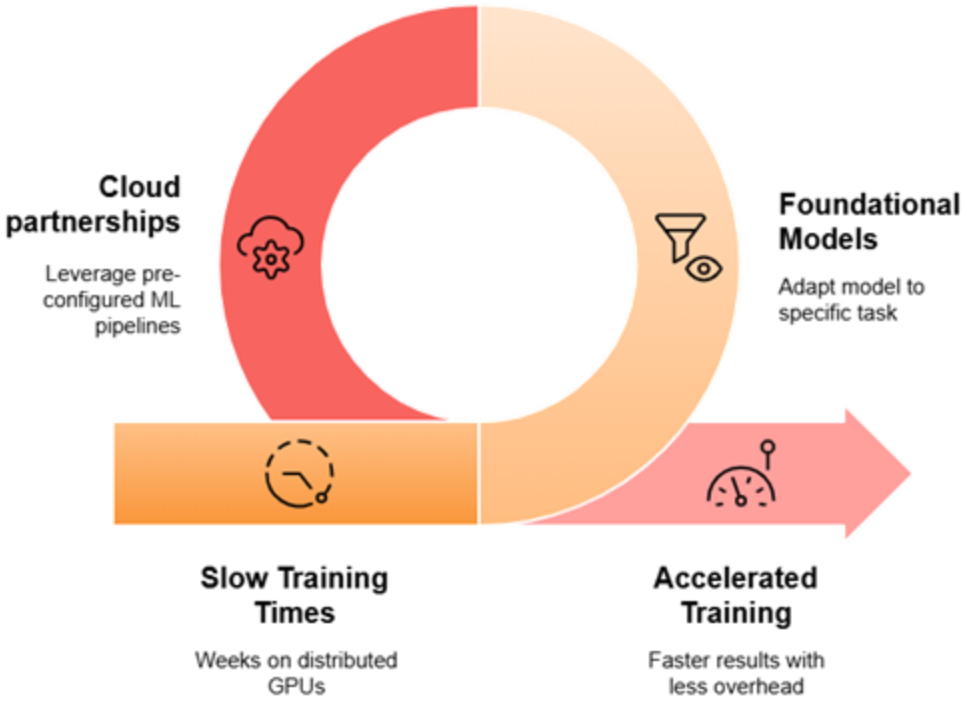
Size and Complexity

- **Challenges:**
  - Specialized **hardware** (A100/H100)
  - LLMs fail **silently** (hallucinations, incoherence)
  - **Difficult** to monitor or evaluate in traditional ways

- **Solutions:**
  - Use **model compression** techniques
  - Apply **structured evaluation protocols**



Training Scale and Duration



Inference Latency and Throughput

- LLM inference is **slow** at scale due to:
  - **large** model **size**
  - **long** prompts
  - **limited** GPU **resources**

- **Solutions:**
  - Use **token streaming** for faster responses
  - **Cache** common **outputs**

User Feedback Loops

Challenges

- LLM output is open-ended
- Hard to evaluate automatically

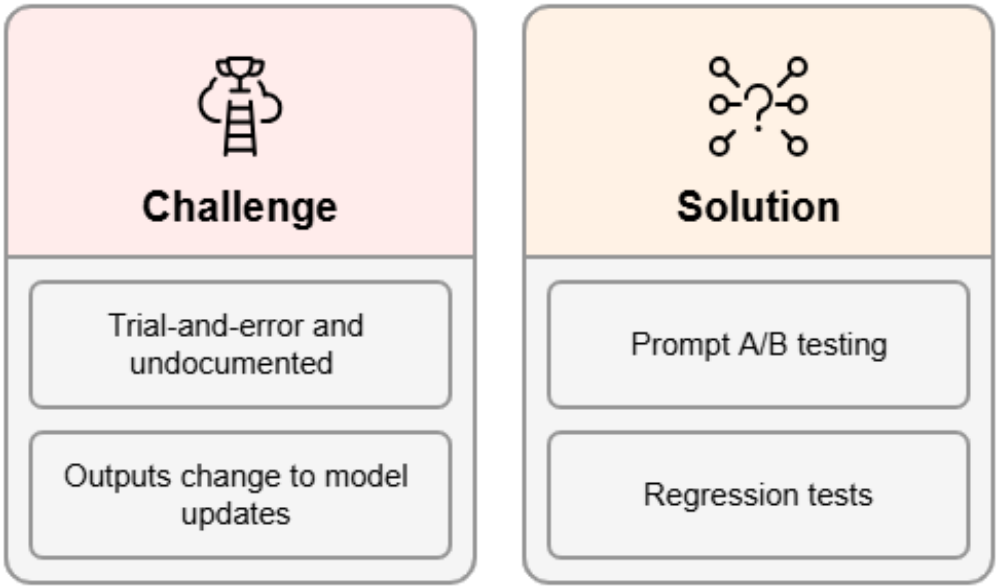
Solutions

- Collect explicit and implicit signals
- Apply RAG pipelines and monitor retrieval/document effectiveness

Costs

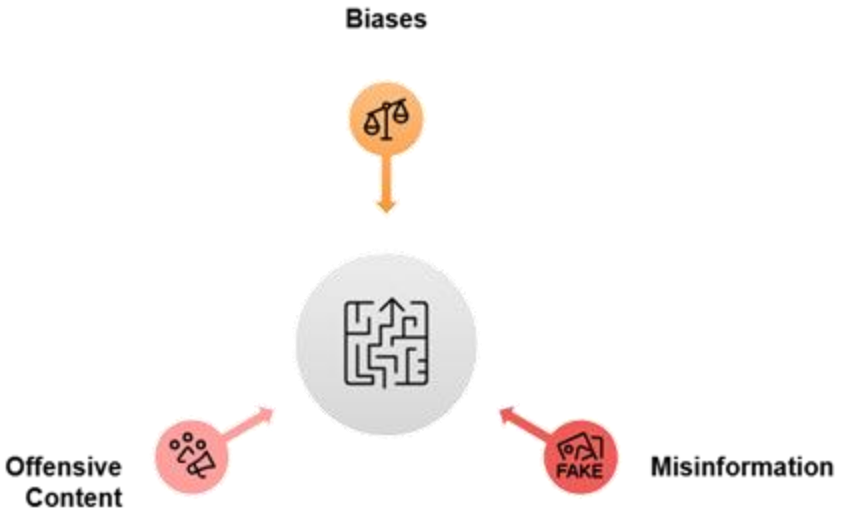
- **Challenge**
  - Training, fine-tuning, inference, and even monitoring are **extremely expensive**
  - Closed models (e.g., GPT-4) **charge** per token
- **Solutions**
  - Use **open-source LLMs** (Mistral, LLaMA 3)
  - Add **cost monitoring dashboards**

Prompt Engineering

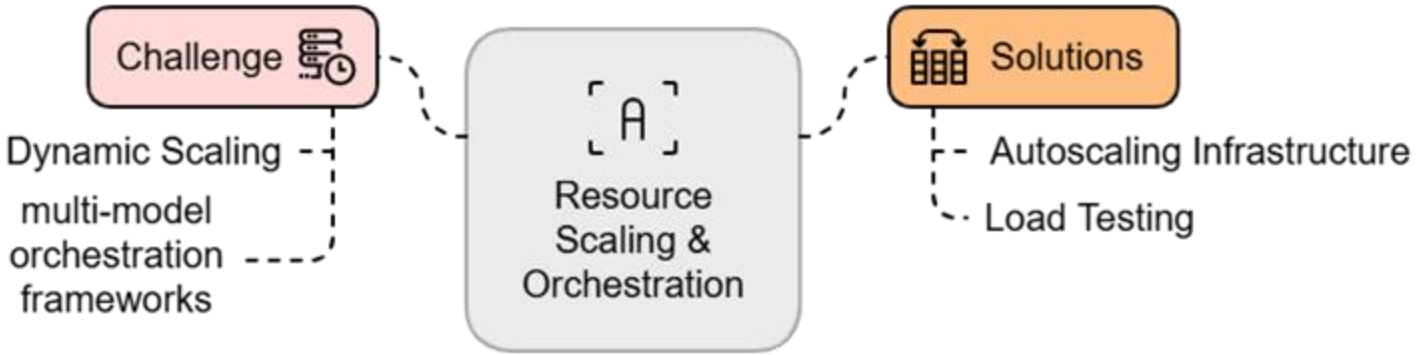


Ethical Considerations

- **Solutions:**
  - Perform **bias audits** using synthetic and real-world test cases
  - In high-risk applications, add **human-in-the-loop review**



Resource Scaling and Orchestration



Summary

Key Takeaways

- LLMs need **powerful** hardware and **smart** scaling.
- Latency and cost can be reduced with **streaming, caching,** and **open-source** models
- **Prompt engineering** and **user feedback** are crucial but complex
- **Ethics, privacy,** and **orchestration** must be built into deployment from day one

Discussion Question

How does LLM deployment differ from smaller models?

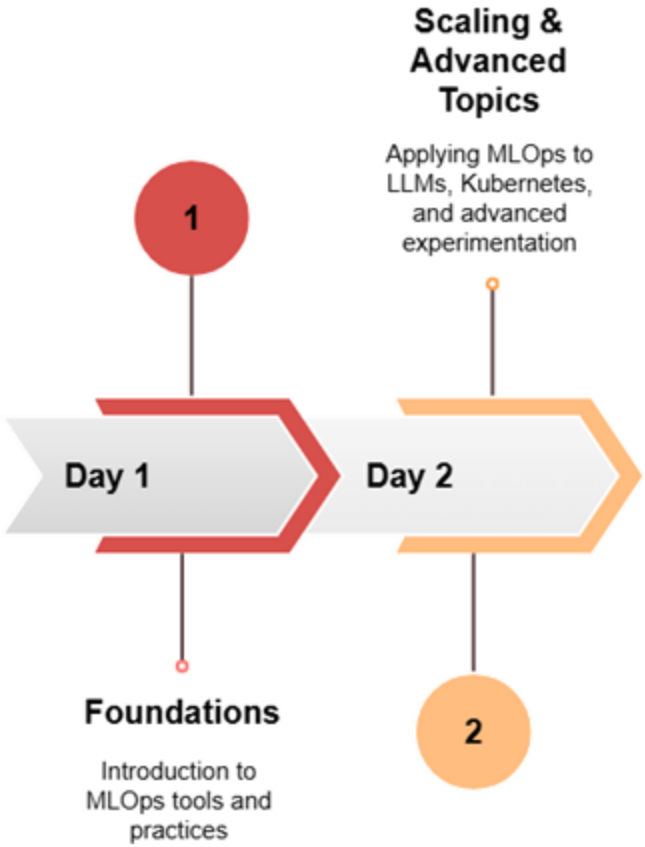

O'REILLY®

O'REILLY

Recap

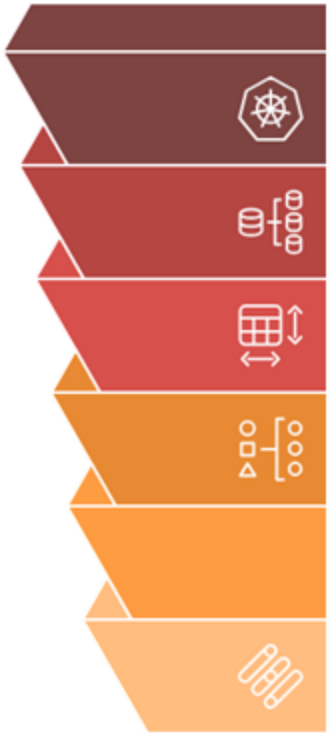
Ammar Mohanna

Overview


Scaling & Advanced Topics
Applying MLOps to LLMs, Kubernetes, and advanced experimentation
1
Day 1
Day 2
Foundations
Introduction to MLOps tools and practices
2

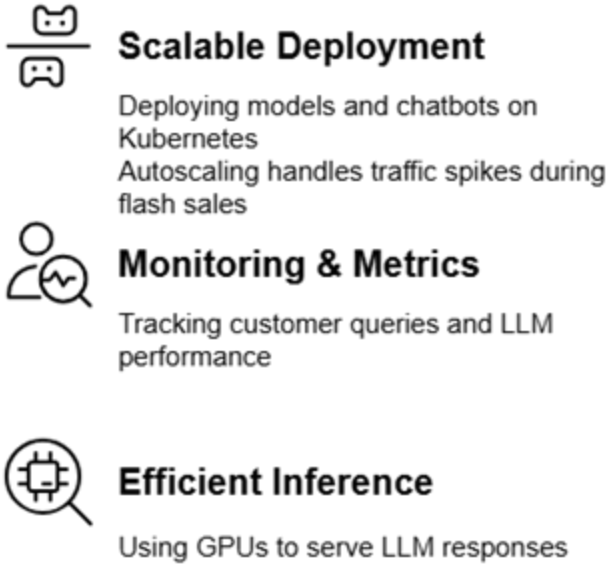
Key Learnings

1. **Kubernetes Deployment**
   Deploying and scaling models with Kubernetes
2. **Model Monitoring**
   Monitoring models for data drift and performance
3. **Scaling Strategies**
   Exploring GPU scaling and high-throughput management
4. **Workflow Automation**
   Automating ML workflows with Kubeflow
5. **Safe Experimentation**
   Conducting A/B testing and canary deployments
6. **Lifecycle Management**
   Managing model lifecycle with registries and controls



Integrating MLOps/LLMOps into Daily Workflows

Example Use Case: E-commerce Product Recommendation System with LLM-based Search Assistant



Integrating MLOps/LLMOps into Daily Workflows

Automated Pipelines

Retrain the recommendation model using fresh user interaction data
Prompts updated weekly

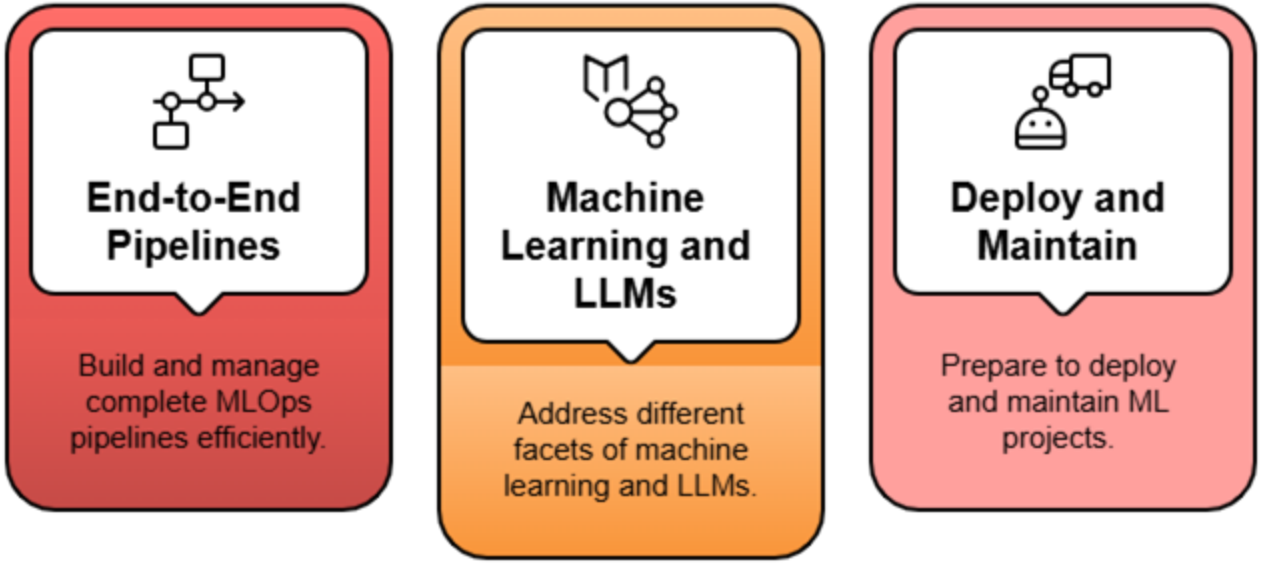
Safe Experimentation

Conducting A/B tests for LLM prompts
Canary deploy the winning prompt

Lifecycle Management

Tracking model versions and metadata

What you are ready for

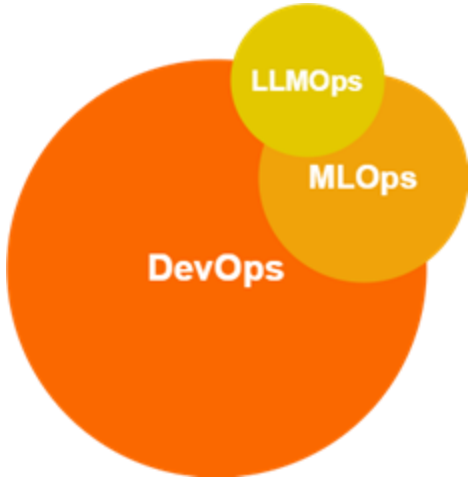


Next Steps in your MLOps Journey

- Keep up with the **latest** industry developments, tools, and methodologies
- Regularly **review** and **refine** your workflows

Conclusion

- MLOps and LLMOps bridge the **gap** between **development** and **production** in AI
- MLOps and LLMOps are key to ensuring **efficiency, scalability,** and **reliability**
- **MLOps** streamlines **machine learning** lifecycles
- **LLMOps** addresses the unique challenges of **large language models**



Stay Curious and Keep on Learning

Thank you

Q & A


O'REILLY®